Software Requirements Specification

for

SciTrend

Scientific Journal Publication Trend Tracking System

Version 1.0

Prepared by

Hoàng Thị Khánh Linh • Nguyễn Huy Phong • Hồ Lê Thiên An • Nguyễn Đình Hải Đăng • Huỳnh Hữu Phúc

Course: SWR302 – SE2041

July 2026

Table of Contents

Software Requirement Specification **1**
Table of Contents **1**
1. Introduction **5**
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Project Scope and Product Features 5
1.4 References 7
2. Overall Description **8**
2.1 Product Perspective 8
2.2 User Classes and Characteristics 19
2.3 Operating Environment 20
2.4 Design and Implementation Constraints 21
2.5 Assumptions and Dependencies 23
3. System Features **24**
3.1 Business Rule 24
3.2 Use Cases 30
4. Data Requirements **55**
4.1 Logical Data Model 55
4.2 Data Dictionary 56
4.3 Report 72
4.4 Data Integrity, Retention, and Disposal 74
5. External Interface Requirements **76**
5.1 User Interfaces 76
5.2 Software Interfaces 77
5.3 Hardware Interfaces 79
5.4 Communications Interfaces 79
6. Quality Attributes **80**
6.1 Usability Requirements 80
6.2 Performance Requirements 80
6.3 Security Requirements 80
6.4 Safety Requirements 81
6.5 Availability Requirements 81
6.6 Robustness Requirements 81
Appendix **82**
A. Member Evaluation Table **82**

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the functional and non-functional requirements for version 1.0 of the SciTrend system.

SciTrend is a web-based application that supports users in searching for, tracking, and managing scientific research publications. The system integrates with the OpenAlex API to provide data about publications, authors, topics, institutions, and other related academic information.

This document is intended for members of the project team, including the Product Owner, Business Analyst, UI/UX Designer, Developers, Testers, and other stakeholders involved in the analysis, design, implementation, testing, and acceptance of the system.

Unless otherwise specified, all requirements described in this document are included in the planned scope of version 1.0.

1.2 Document Conventions

No special typographical conventions are used in this SRS.

1.3 Project Scope and Product Features

SciTrend is a web-based system that supports the discovery and analysis of scientific research information. The system retrieves academic data directly from the OpenAlex API, allowing users to search for research publications, view publication details, track publication trends, and save important references.

The primary objective of SciTrend is to help Researchers discover relevant academic publications, follow authors and topics of interest, manage saved research collections, review their search history, and export data for analysis or sharing.

1.3.1 Features Included in Version 1.0

Version 1.0 of SciTrend includes the following features:

- User authentication and authorization.
- User registration, login, email verification, and password recovery.
- Google OAuth2 login.
- Research publication search.
- Publication detail viewing.
- Publication trend tracking.
- Dashboard data visualization.
- Bookmark and bookmark-collection management.
- Searching, filtering, and sorting bookmarked papers.
- Adding personal notes to bookmarks.
- Following authors and topics.
- Saving and viewing search history.

- OpenAlex API integration.
- Creating, viewing, editing, and deleting personal posts in the Social Hub.
- Sharing bookmarked papers to the Social Hub.
- Basic administration functions, including viewing the Admin Dashboard and banning or unbanning user accounts.
- Sending email notifications when an account is banned or unbanned.
- Exporting bookmarked-paper data in CSV or JSON format.
- Allowing users to choose which metadata fields are included in an export.

1.3.2 Features Excluded from Version 1.0

The following features are outside the scope of version 1.0:

- AI Research Trend Assistant.
- Advanced recommendation engine.
- Co-authorship network analysis.
- A dedicated mobile application.
- A responsive interface optimized for mobile phones and tablets.
- Real-time notifications.

1.3.3 Planned Features for Future Releases

Version 2.0 and later releases may include:

- AI Research Trend Assistant.
- Automated trend reports.
- Topic recommendation engine.
- Research gap analysis.
- Email notifications for following authors or topics.
- Advanced bibliometric analytics.
- Collaboration-network visualization.
- A mobile-friendly interface.
- Integration with multiple academic databases.

1.3.4 Long-Term Improvements

Long-term improvements may include:

- Predictive research-trend forecasting.
- Research-impact prediction.
- Institution-level analytics.
- A personalized AI research assistant.

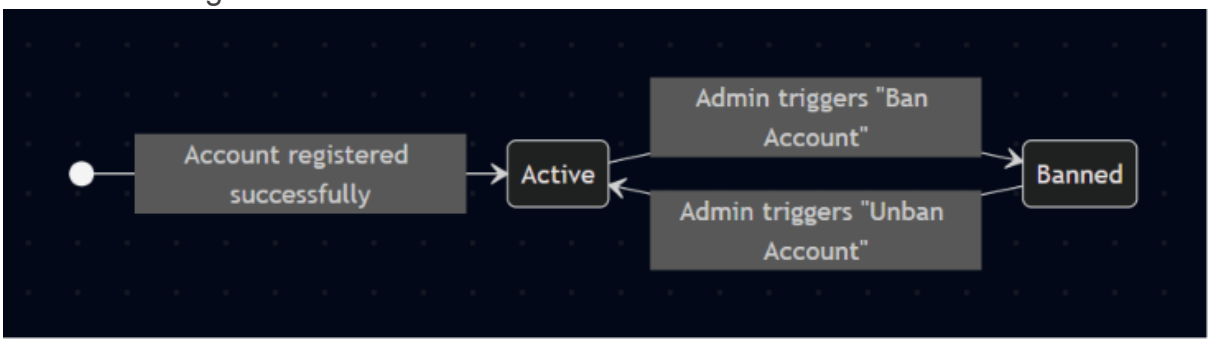
1.4 References

SciTrend Project Team, *SciTrend Vision and Scope Document*, Version 1.0, 2026.

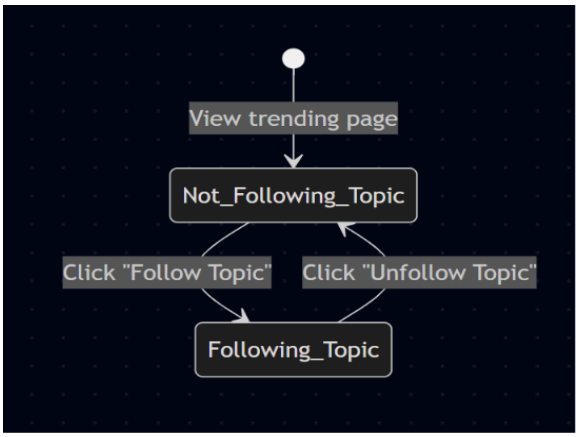
2. Overall Description

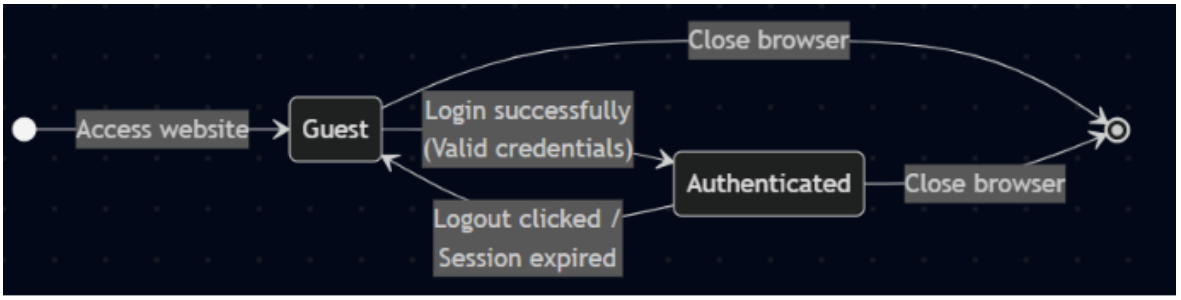
2.1 Product Perspective

2.1.1 State diagram





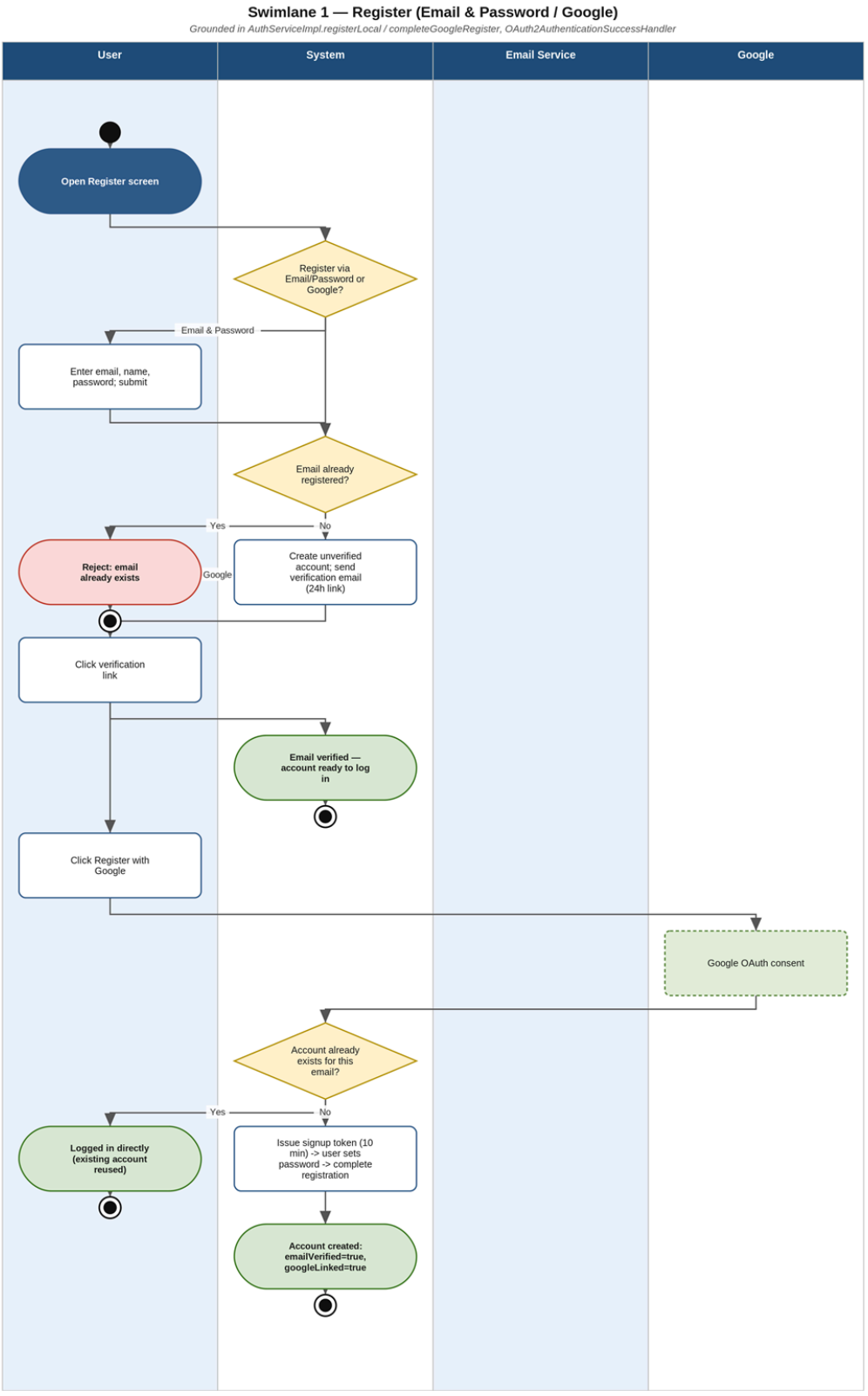




2.1.2 Use Case Diagram

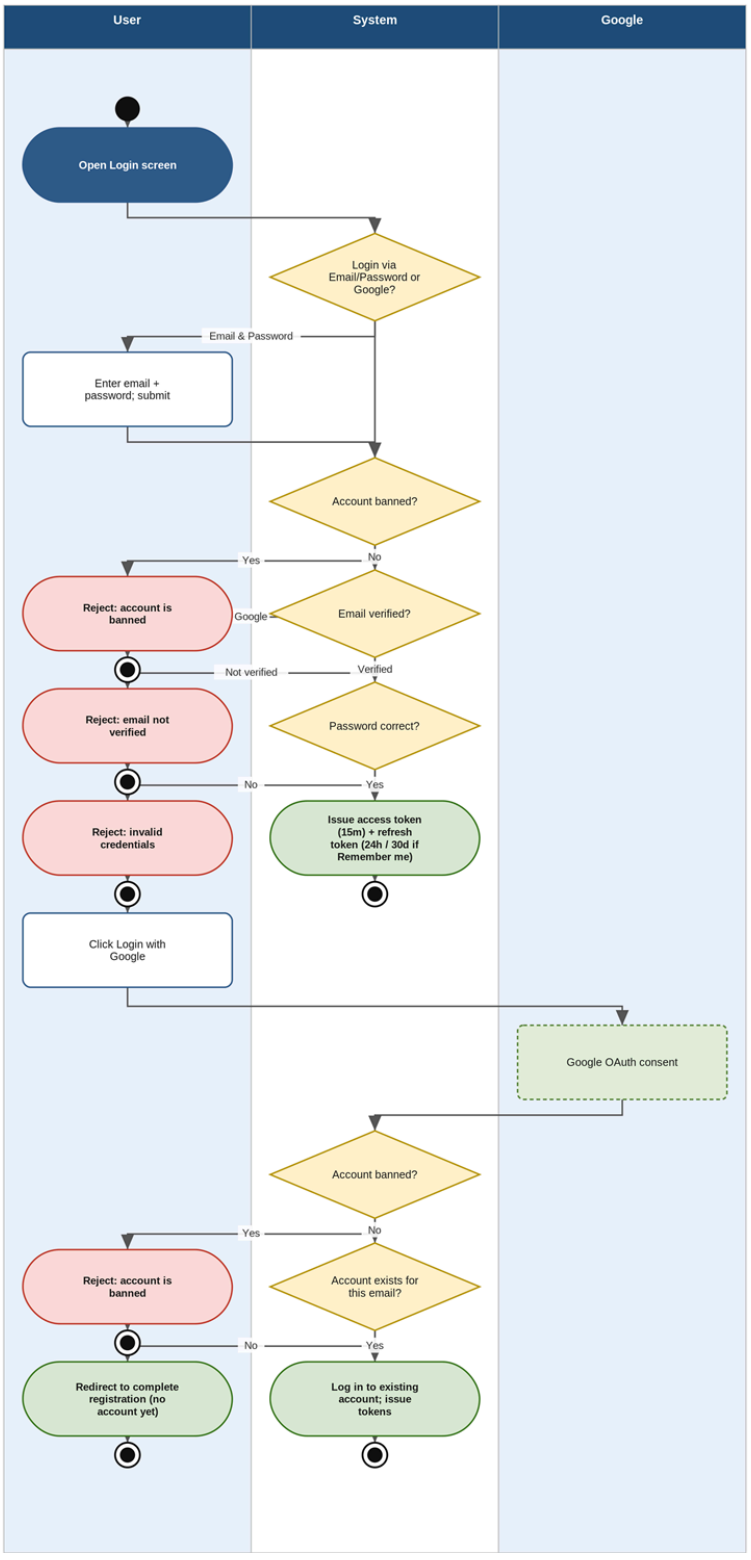

System
View User Detail
Ban user
<<include>>
<<extend>>
<<extend>>
Send Notification Email
Manage Users
<<extend>>
<<include>>
Unban user
View System Statistical Reports
Manage Update Time Configuration
Verify OTP
<<include>>
Forgot Password
Verify Email
<<include>>
Register
<<extend>>
Google OAuth Login
Manage Academic Bookmarks
View Entity Details
<<extend>>
Search Academic Entities
Retrieve Bookmark Collections
<<include>>
Export Paper Report
Manage User Profile
Change Password
Manage Social Posts
View Weekly Data Analysis
Manage Followed Entities
Admin
User - Researcher
Open Alex
Gmail SMTP
Google OAuth 2.0
Authenticate with Google
DiceBear

2.1.3 Swimlane Flowchart



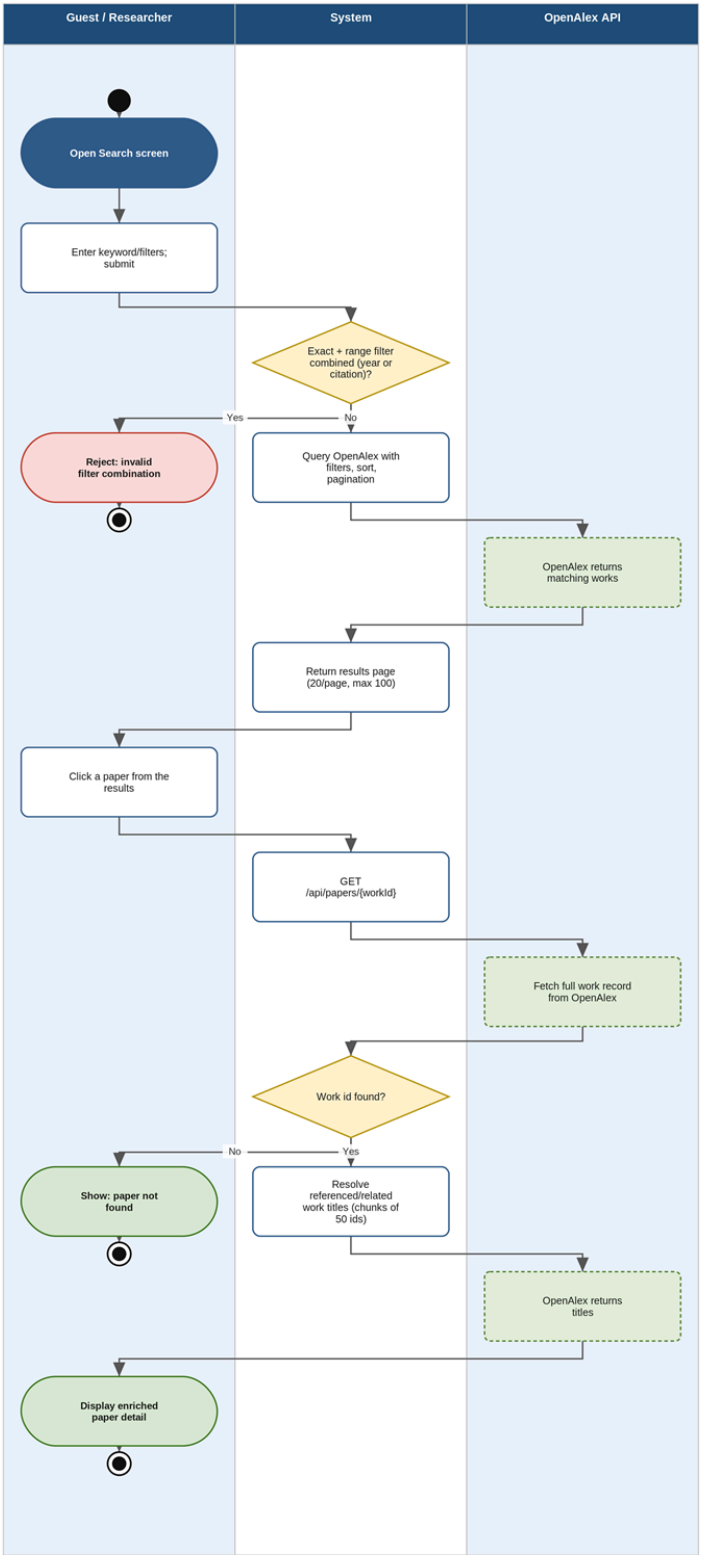
Swimlane 2 — Login (Email & Password / Google)

Grounded in AuthServiceImpl.login, CustomOAuth2UserService, OAuth2AuthenticationSuccessHandler



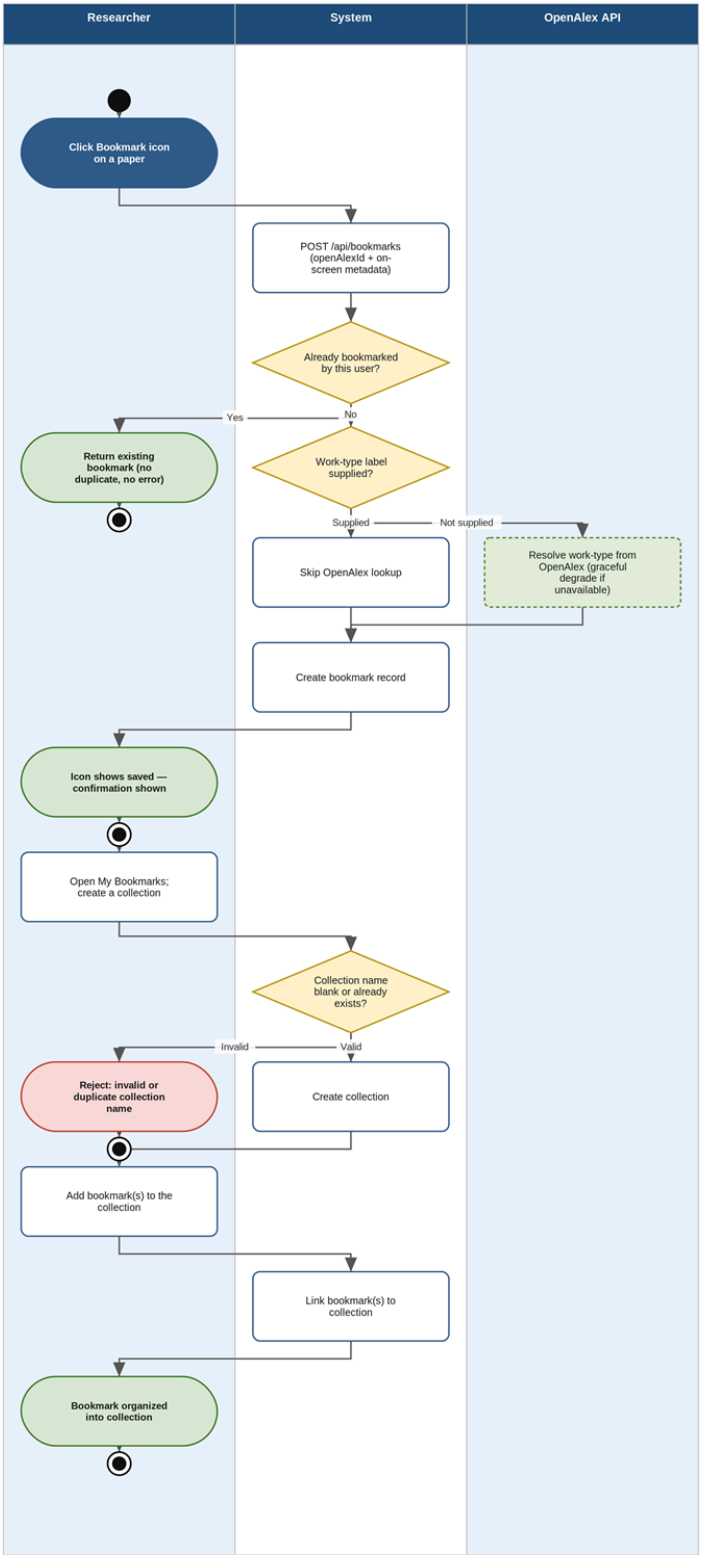
Swimlane 3 — Search Papers & View Paper Detail

Grounded in SearchController, SearchWorksLookupService, PaperDetailServiceImpl



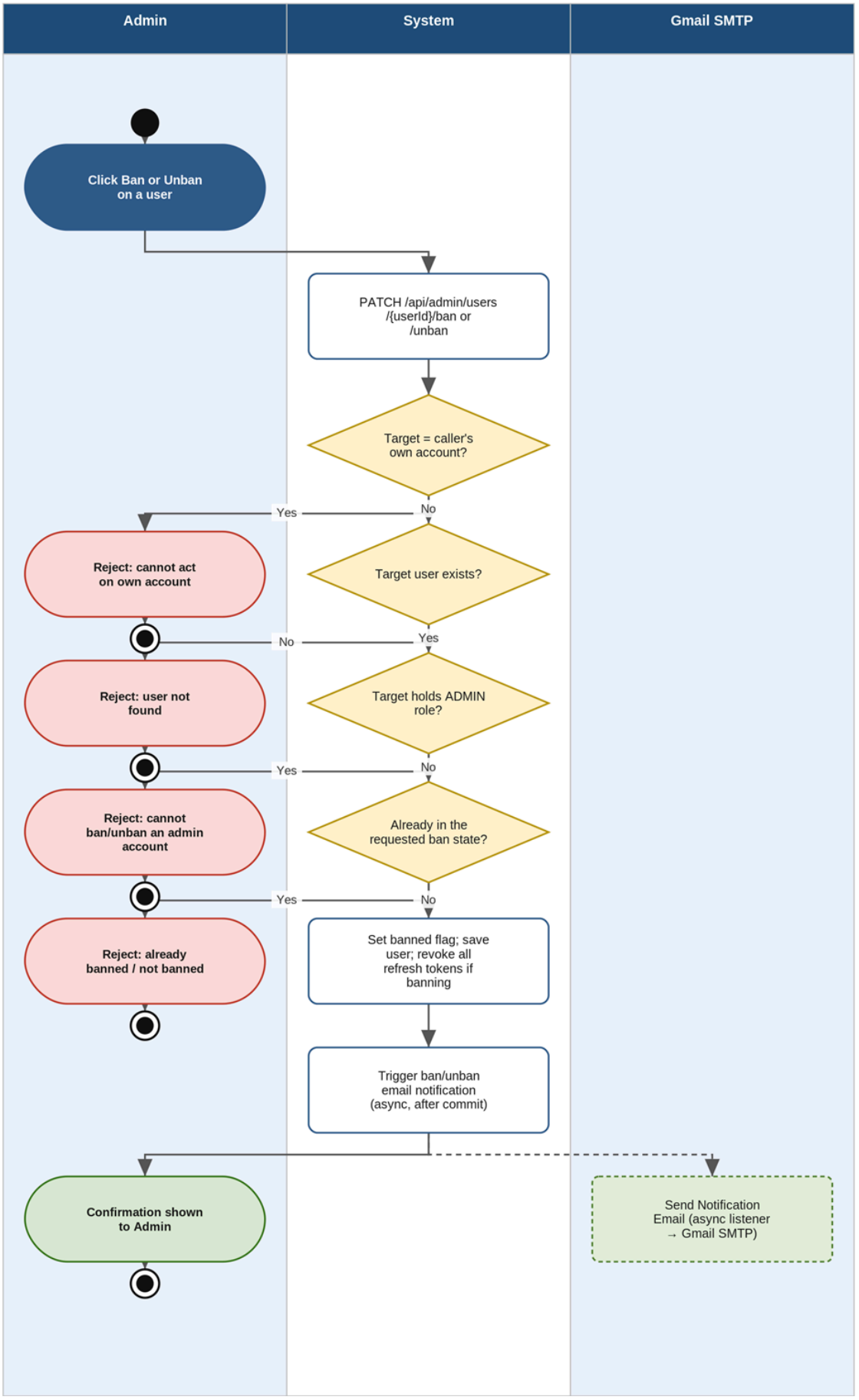
Swimlane 4 — Save and Manage Bookmarked Papers

Grounded in BookmarkServiceImpl (addBookmark, createCollection, addBookmarksToCollection)



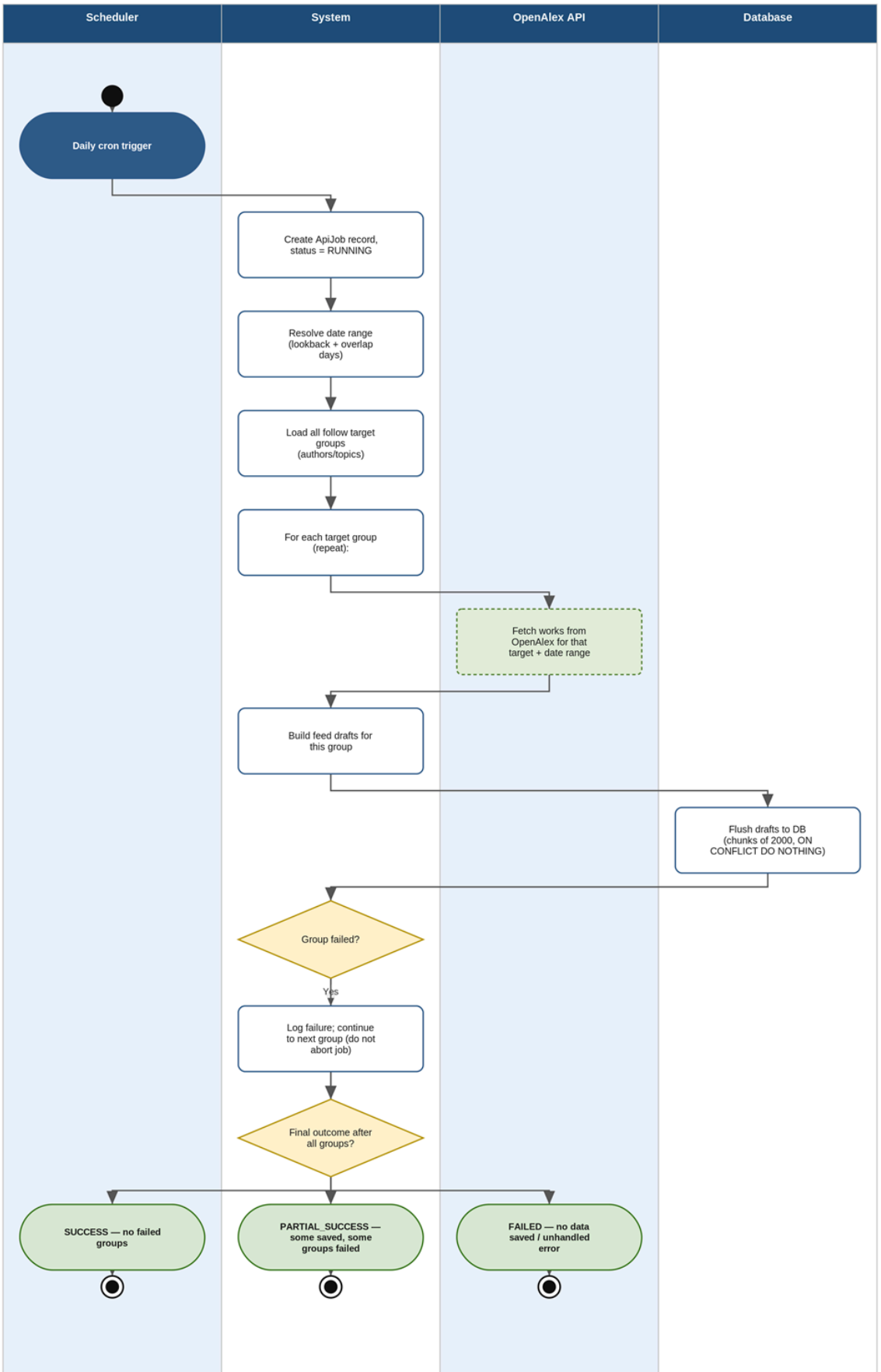
Swimlane 5 — Ban / Unban a User Account

Grounded in AdminController.banUser/unbanUser, AdminServiceImpl.getValidTargetUser, UserAccountStatusMailListener



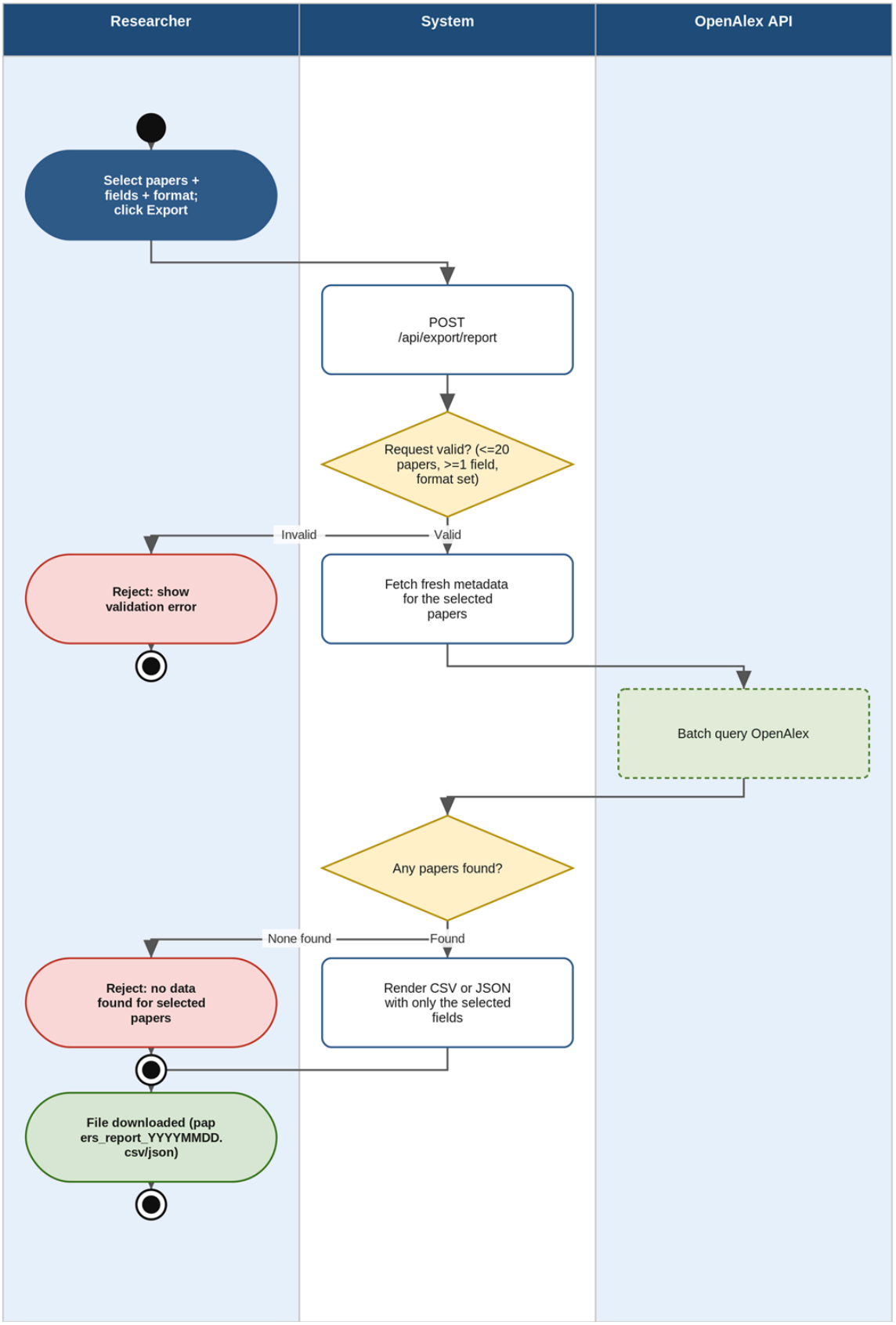
Swimlane 6 — Personalized Feed Sync (scheduled job)

Grounded in ResearchFeedSyncServiceImpl.syncDailyFeed (chunked/batched design, ApiJob status tracking)



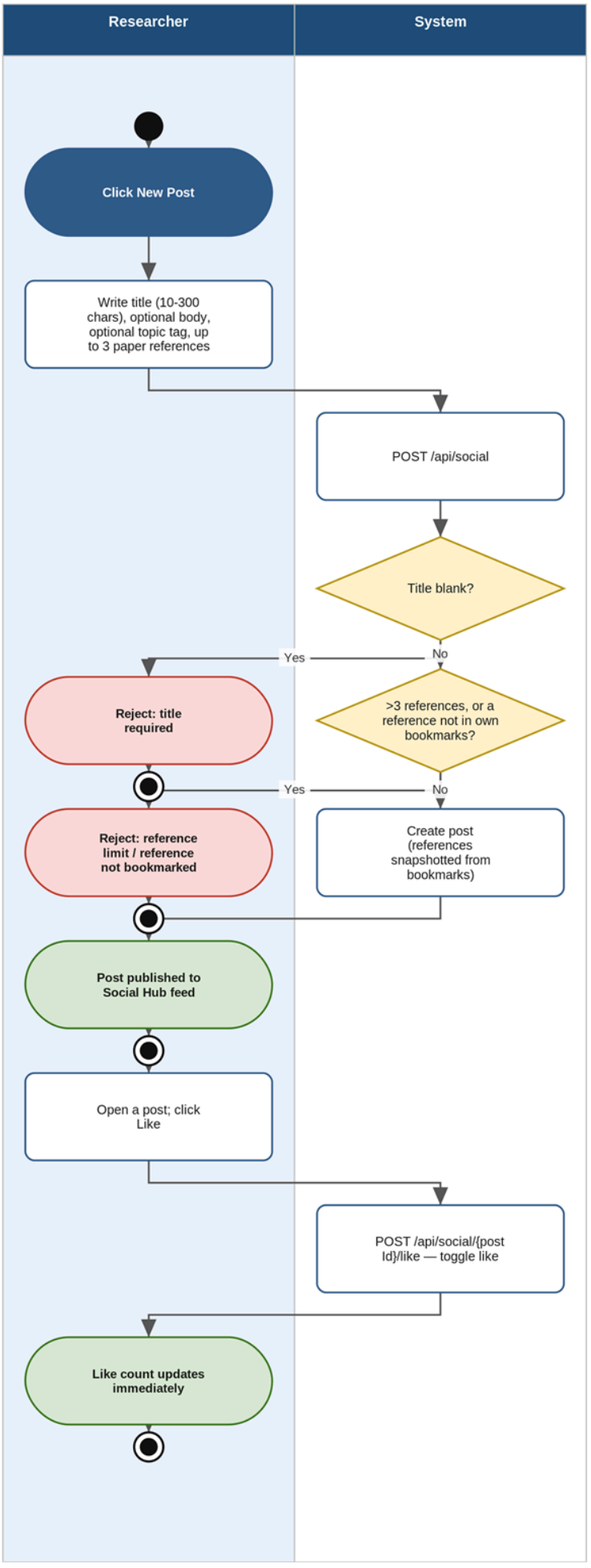
Swimlane 7 — Export Bookmarked Papers (Report)

Grounded in ExportReportServiceImpl, PaperExportDataServiceImpl



Swimlane 8 — Social Hub: Create Post & Like

Grounded in SocialServiceImpl.createPost / toggleLike, CreateSocialPostRequest



2.2 User Classes and Characteristics

Academic User (Student / Lecturer / Researcher)	An Academic User is a registered user affiliated with a university or research institution who uses SciTrend to monitor scientific publication trends and discover research topics. They can search for papers by work, author, or topic; view publication metadata; analyze trend charts and dashboards; bookmark papers; follow authors and topics; receive notifications for newly published papers; and export reports in CSV or JSON format. They have high expectations for data accuracy and require a registered account for full access.
Guest	A Guest is an unregistered visitor who accesses SciTrend without logging in. Guests can perform basic keyword searches, view publicly available trend charts, and browse publication details without creating an account. However, they cannot save bookmarks, follow authors or topics, receive notifications, or access personalized dashboards. Guests are typically casual visitors evaluating the platform prior to registration.
System Administrator	A System Administrator is a technical staff member responsible for maintaining the SciTrend platform. Administrators manage user accounts, assign roles, configure system settings, monitor performance, and oversee scheduled data synchronization from the OpenAlex API. A small number of Administrators (estimated 1–3) will access the system as needed and require full access to all backend controls.

2.3 Operating Environment

The system is developed as a web application and is primarily intended for use on desktop computers.

2.3.1 Client Environment

- The frontend is developed using React and TypeScript.
- The system operates on modern web browsers such as Google Chrome and Microsoft Edge.
- The system supports Windows, Linux, and macOS.
- The user interface is currently designed for desktop screens only and does not support responsive layouts for mobile phones or tablets.
- Users must have an Internet connection to access the system and use external services.

2.3.2 Server Environment

- The backend is developed using Spring Boot and Java.
- The backend application is deployed on Azure App Service.
- The backend runtime environment uses JDK 25 and Maven.
- Azure App Service provides HTTPS connections for the system.
- The system uses Spring Actuator to support application health monitoring.

2.3.3 Database Environment

- The system uses PostgreSQL as its primary database.
- The database is hosted on the Supabase Cloud platform.

- Database schema changes are managed using Flyway and are executed automatically when the application starts.

2.3.4 External Service Environment

To operate fully, the system requires an Internet connection to access the following services:

- OpenAlex API for retrieving research papers, authors, topics, and research metrics.
- Google OAuth2 for user registration and login using Google accounts.
- Gmail SMTP for sending account verification emails and password recovery OTP codes.
- DiceBear for generating default user avatars.

2.4 Design and Implementation Constraints

The system shall be designed and implemented according to the following constraints:

2.4.1 Technology Constraints

- The frontend shall be developed using React and TypeScript.
- The backend shall be developed using Spring Boot and Java.
- PostgreSQL shall be used as the primary database.
- The frontend and backend shall communicate through REST APIs using JSON.
- Docker shall not be used in the current deployment process.
- The backend shall run in a JDK 25 environment and use Maven for dependency and build management.

2.4.2 Architecture Constraints

- The backend shall follow an N-Layer Architecture consisting of the following layers: **Controller → Service → Repository → Database**
- The frontend and backend shall be developed as separate components.
- The frontend shall not access the database directly.
- All business operations and data access shall be handled through backend APIs.
- The Controller layer shall receive requests and return responses.
- The Service layer shall handle business logic.
- The Repository layer shall retrieve and update data.

2.4.3 Security Constraints

- The system shall use JWT access tokens for user authentication.
- Refresh tokens shall be stored in HttpOnly cookies.
- The system shall support Google account login through OAuth2.
- The system shall provide two main roles: RESEARCHER and ADMIN.
- Protected endpoints shall validate the JWT and the user’s permissions before processing requests.
- Invalid, unauthenticated, or unauthorized requests shall be rejected with appropriate HTTP status codes.
- Communication between the client and server in the deployment environment shall use HTTPS.

2.4.4 Database Constraints

- Database schema changes shall be managed using Flyway migrations.
- Migrations shall be executed automatically when the backend starts.
- The backend shall access PostgreSQL through Spring Data JPA.
- The frontend shall not connect directly to Supabase or PostgreSQL.

2.4.5 Development Constraints

- The source code shall be managed on GitHub.
- Each feature shall be developed on a separate feature branch.
- A Pull Request shall be created before code is merged into the dev branch.
- Pull Requests shall be checked and reviewed before being merged.
- CodeRabbit shall be used to automatically analyze and review Pull Requests.
- Sensitive information, including database credentials, JWT secrets, OAuth client secrets, and SMTP passwords, shall not be hard-coded in the source code.

2.4.6 Deployment Constraints

- The backend shall be deployed on Azure App Service.
- The deployment process shall be integrated with GitHub.
- After the CI process succeeds and the source code is merged into the deployment branch, the system shall automatically build and deploy the new version.
- Deployment shall not require direct modification of the source code on the server.
- Docker containers shall not be used in the current environment.

2.4.7 Integration Constraints

- The system shall use the OpenAlex API as its main academic data source.
- The system shall use Google OAuth2 for Google account login.
- The system shall use Gmail SMTP to send account verification emails and OTP codes.
- The system shall use DiceBear to generate default avatar URLs.
- When an external service is unavailable, the backend shall handle the error and return an appropriate response instead of causing the entire system to become unavailable.

2.4.8 Completion Constraints

A feature shall only be considered completed when:

- The backend returns a JSON response with the correct structure.
- The feature can be verified through Swagger UI or Postman.
- JWT authentication works correctly.
- Authorization between the RESEARCHER and ADMIN roles works correctly.
- Invalid or unauthorized requests return appropriate HTTP status codes.
- The feature satisfies the agreed Acceptance Criteria.

2.5 Assumptions and Dependencies

2.5.1 Assumptions

AS-1: Users have a stable Internet connection to access the system and view analytical data.

AS-2: Daily published academic papers provide sufficient data volume to calculate statistically meaningful publication trends and keyword frequency analysis.

AS-3: Users enter valid keywords, research fields, or time ranges when performing searches and trend analysis.

AS-4: Quantitative indicators such as publication count, citation count, authors, institutions, and research topics can be used as reliable metrics for evaluating research trends.

AS-5: Target users actively prefer visual tools such as dashboards and charts over text-based reading to support research topic selection and trend identification.

2.5.2 Dependencies

DE-1: The operation of SciTrend depends on the continued availability, stability, and data accuracy of the OpenAlex API for fetching publication data including papers, authors, institutions, research fields, publication years, and citation counts.

DE-2: If the OpenAlex API experiences downtime, structural changes, or rate limit restrictions, the system may be unable to retrieve updated data or may produce incomplete trend analysis results.

DE-3: The trend visualization features depend on high-performance web charting libraries to render interactive dashboards and publication trend charts accurately and efficiently.

DE-4: The system depends on an internal database to cache data retrieved from OpenAlex, store user search history, and persist analytical results for improved performance and reliability.

3. System Features

3.1 Business Rule

A. User Account & Authentication

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-1	Local registration requires a unique email address, first name, last name, and a password (all non-blank).	Constraint	Static	Source code: RegisterLocalRequest.java; AuthServiceImpl.registerLocal
BR-2	Local registration requires Password and Confirm Password to match and to be between 10 and 100 characters, mirroring the validation already enforced for the Google-completion flow (BR-17).	Constraint	Static	Source code: RegisterLocalRequest.java / AuthServiceImpl.registerLocal; mirrors CompleteGoogleRegisterRequest.java for the Google-completion flow (BR-17).
BR-3	A newly registered local account is created unverified and cannot log in until its email address is verified.	Constraint	Static	Source code: AuthServiceImpl.registerLocal / login (EMAIL_NOT_VERIFIED)
BR-4	An email verification link expires 24 hours after it is issued.	Constraint	Dynamic	Source code: AuthServiceImpl.registerLocal
BR-5	A banned account (is_banned = true) is denied login through both the local and Google sign-in paths.	Constraint	Static	Source code: AuthServiceImpl.login; CustomOAuth2UserService; OAuth2AuthenticationSuccessHandler
BR-6	The system defines exactly two roles, RESEARCHER and ADMIN; every account has exactly one role.	Fact	Static	Source code: Role.java; users.role

BR-7	A JWT access token is valid for 15 minutes by default and must be renewed via the refresh-token endpoint.	Fact	Dynamic	Source code: JwtTokenService (app.jwt.access-token-minutes)
BR-8	A refresh token is valid for 24 hours by default, or 30 days when "Remember me" is selected at login; it is stored only in an HttpOnly cookie.	Fact	Dynamic	Source code: AuthProperties; RefreshCookieService
BR-9	Each time a refresh token is used to obtain a new access token, it is rotated: the old refresh token is revoked and a new one is issued.	Constraint	Dynamic	Source code: RefreshTokenServiceImpl.rotate
BR-10	Presenting a refresh token that has already been rotated (revoked) is treated as a token-theft signal: the system immediately revokes every active refresh token belonging to that user.	Constraint	Dynamic	Source code: RefreshTokenServiceImpl.rotate (reuse detection)
BR-11	Logging out revokes all of the user's active refresh tokens, signing the user out of every device/session, not only the current one.	Constraint	Static	Source code: AuthServiceImpl.logout; RefreshTokenService.revokeAllByUserId
BR-12	A user may hold multiple concurrent active sessions across different devices, since logging in does not revoke previously issued refresh tokens.	Fact	Static	Source code: RefreshTokenServiceImpl.issue
BR-13	Google sign-in reuses the existing account when the Google email matches an existing user; it never creates a duplicate account for an email that already exists.	Fact	Static	Source code: OAuth2AuthenticationSuccessHandler
BR-14	Google sign-in is accepted only if Google reports the account's email as verified (email_verified = true); otherwise it is rejected.	Constraint	Static	Source code: CustomOAuth2UserService
BR-15	When no account exists for the Google email, the system does not create one automatically; it issues a one-time Google signup token and routes the user to a "complete registration" step.	Constraint	Dynamic	Source code: OAuth2AuthenticationSuccessHandler
BR-16	A Google signup token can be redeemed only once and expires 10 minutes after issuance.	Constraint	Dynamic	Source code: GoogleSignupToken.java
BR-17	Completing a Google registration requires Password and Confirm Password to match and to be between 10 and 100 characters.	Constraint	Static	Source code: CompleteGoogleRegisterRequest.java; AuthServiceImpl.completeGoogleRegister
BR-18	An account created by completing Google registration is marked emailVerified = true and googleLinked = true immediately, skipping the local email-verification step.	Fact	Static	Source code: AuthServiceImpl.completeGoogleRegister
BR-19	A password-reset OTP code is 6 digits long, expires 10 minutes after issuance, and may be attempted a maximum of 5 times before it is invalidated.	Constraint	Dynamic	Source code: AuthProperties; PasswordRecoveryServiceImpl
BR-20	Requesting a password reset always returns the same generic confirmation, whether or not the email is registered or is Google-only, to prevent account enumeration.	Constraint	Static	Source code: PasswordRecoveryServiceImpl
BR-21	Every protected API endpoint validates the caller's JWT and role before processing the request; unauthenticated or unauthorized requests are rejected with the appropriate HTTP status code.	Constraint	Static	Source code: JwtAuthenticationFilter; SecurityConfig
BR-22	All requests carrying credentials or personal data must be transmitted over HTTPS.	Constraint	Static	SRS 6.3 Security Requirements (SEC-3)

B. Bookmark & Collections

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-23	Only authenticated, non-banned users may create, view, modify, or delete bookmarks and bookmark collections — and only their own, never another user's. (Merged with the former BR-26.)	Constraint	Static	Source code: BookmarkController (@CurrentUserUUID requires an authenticated principal); BookmarkServiceImpl.requireCollection / deleteBookmark (user-scoped queries)
BR-24	A user may have at most one bookmark per OpenAlex work (unique on user_id + openalex_id); a repeated bookmark request for the same paper returns the existing bookmark rather than creating a duplicate or raising an error.	Constraint	Static	Source code: UserBookmark unique constraint; BookmarkServiceImpl.addBookmark
BR-25	A bookmark collection name must be non-blank, at most 120 characters, and unique per user (case-insensitive).	Constraint	Static	Source code: CreateBookmarkCollectionRequest; BookmarkServiceImpl.createCollection
BR-26	Merged into BR-23 (see above).	—	—	—
BR-27	Deleting a bookmark removes it from every collection it belonged to; deleting a collection removes only that collection and its bookmark associations, never the bookmarks themselves.	Fact	Static	Database migration: V14__bookmark_collections.sql (ON DELETE CASCADE)
BR-28	If a bookmark's work-type label is not supplied by the client, the system attempts to resolve it from the OpenAlex API; if OpenAlex cannot be reached, the bookmark is still saved without a resolved work-type label instead of failing.	Constraint	Dynamic	Source code: BookmarkServiceImpl.fetchWorkTypeLabels
BR-29	A bookmark or follow list query returns 12 results per page by default and never more than 50 per page.	Constraint	Static	Source code: BookmarkServiceImpl / FollowServiceImpl (DEFAULT_SIZE / MAX_SIZE)
BR-30	A Researcher may select at most 20 bookmarked papers in a single "Export Bookmarked Papers" report request.	Constraint	Static	Source code: ErrorCode.EXPORT_EXCEEDS_PAPER_LIMIT

C. Follow

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-31	Only authenticated, non-banned users may follow or unfollow an author or a topic.	Constraint	Static	Source code: FollowController
BR-32	A follow's target_type must be AUTHOR or TOPIC; following a source/journal is not supported in the current data model.	Constraint	Static	Source code: FollowTargetType.java
BR-33	A user may follow the same target only once (unique on user_id + target_type + target_openalex_id); a repeated follow request returns the existing follow rather than creating a duplicate or raising an error.	Constraint	Static	Source code: UserFollow unique constraint; FollowServiceImpl.followTarget
BR-34	Unfollowing a target the user does not currently follow is treated as a no-op success.	Fact	Static	Source code: FollowServiceImpl.unfollowTarget
BR-35	A user may filter and sort only their own list of followed authors/topics (backend-scoped by userId); the backend API supports keyword and target-type filters plus sort, but the production frontend does not currently expose filter/sort controls, offering view only.	Constraint	Static	Source code: FollowController.getMyFollows / FollowServiceImpl.getMyFollows (keyword, targetType, sort params, userId-scoped).

D. Personalized Feed

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-36	The scheduled FEED_SYNC job builds each user's personalized feed exclusively from the authors and topics that the user currently follows.	Fact	Dynamic	Source code: ResearchFeedSyncServiceImpl; UserFollowRepository
BR-37	Feed synchronization processes and persists results in small batches (per follow-target group, chunked by follower) to keep memory usage bounded on limited server resources.	Fact	Dynamic	Source code: ResearchFeedSyncServiceImpl (chunked/batched design)
BR-38	Each feed synchronization run is logged as an api_job with a status of RUNNING, SUCCESS, PARTIAL_SUCCESS, or FAILED.	Fact	Static	Source code: ApiJob.java; ResearchFeedSyncServiceImpl

E. Trend Analysis & Dashboard

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-39	A topic's trend status is computed from its period-over-period change: BREAKOUT (>= 100%), HOT (>= 50%), or RISING (below 50%).	Computation	Dynamic	Source code: TopicStatus.fromChange()
BR-40	Every displayed trend-analysis result must note that it is for reference only and depends on the completeness of OpenAlex data.	Constraint	Static	SRS 6.4 Safety Requirements (SAF-1); Vision & Scope LI-3

F. Reporting / Export

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-41	An export request must include at least one paper, at least one metadata field, and a chosen format (CSV or JSON); the system rejects requests missing any of these.	Constraint	Static	Source code: ErrorCode.EXPORT_NO_PAPER_SELECTED / EXPORT_NO_FIELD_SELECTED / EXPORT_FORMAT_REQUIRED
BR-42	An export request may include at most 20 papers.	Constraint	Static	Source code: ErrorCode.EXPORT_EXCEEDS_PAPER_LIMIT
BR-43	The system supports 15 exportable metadata fields (Title, Authors, Year, Citation Count, Open Access, Field, Domain, Keyword, Subfield, Topic, Institution, Country, DOI, Abstract, Type); only the fields the user explicitly selects are included in the output.	Fact	Static	Source code: ExportField.java

G. Social Hub

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-44	A Social Hub post may reference at most 3 papers.	Constraint	Static	Source code: ErrorCode.SOCIAL_POST_EXCEEDS_REFERENCE_LIMIT
BR-45	Every paper referenced in a Social Hub post must already exist in the author's own bookmark list.	Constraint	Static	Source code: ErrorCode.SOCIAL_POST_REFERENCE_NOT_IN_BOOKMARK
BR-46	A Social Hub post's title cannot be blank; the body is optional with no blank or length requirement.	Constraint	Static	Source code: CreateSocialPostRequest.java (title: @NotBlank @Size(10,300); body: no validation annotation); SocialServiceImpl.createPost (only title checked, via StringUtils.hasText)

H. Administration

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-47	Every endpoint under /api/admin/** is restricted to authenticated users holding the ADMIN role; this is enforced declaratively at the security-filter level, not inside the admin controller/service.	Constraint	Static	Source code: SecurityConfig.java (.requestMatchers("/api/admin/**").hasRole("ADMIN"))
BR-48	An Administrator cannot ban or unban their own account, and Administrator accounts can never be targeted for ban/unban.	Constraint	Static	Source code: ErrorCode.ADMIN_SELF_ACTION_NOT_ALLOWED / ADMIN_TARGET_NOT_ALLOWED
BR-49	Banning a user immediately revokes all of that user's active refresh tokens (signing them out everywhere) and sends a notification email.	Action Enabler	Dynamic	Source code: AdminServiceImpl.banUser; UserBannedEvent
BR-50	Unbanning a user sends a notification email confirming that account access has been restored.	Action Enabler	Static	Source code: AdminServiceImpl.unbanUser; UserUnbannedEvent
BR-70	The admin user list defaults to 10 accounts per page and never returns more than 100; it may be sorted by Most Recent (default), Oldest, Email A-Z, or Email Z-A.	Constraint	Static	Source code: AdminServiceImpl.normalizeSize / buildUserSort

I. Data Integration (OpenAlex Synchronization)

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-51	All OpenAlex API calls are retried automatically up to 5 times with exponential backoff for retryable errors (HTTP 429 or 5xx); a 404 is treated as "entity not found" and is not retried.	Constraint	Dynamic	Source code: common/openalex/OpenAlexClient.java
BR-52	Wherever a feature has been designed to tolerate it, an unavailable OpenAlex API degrades that feature gracefully (e.g., a bookmark is still saved without a resolved work-type label) instead of failing the whole request.	Constraint	Dynamic	Source code: BookmarkServiceImpl.fetchWorkTypeLabels (example)

J. System Quality (Security, Performance, Availability, Robustness, Safety)

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-53	Users must log in to save search history, create reports, bookmark papers, or follow authors/topics.	Constraint	Static	SRS 6.3 Security Requirements (SEC-1)
BR-54	Each user may view and download only the reports/analyses they created themselves.	Constraint	Static	SRS 6.3 Security Requirements (SEC-2)
BR-55	The system must verify the caller's permissions before allowing deletion of a report or viewing another user's personal analysis history.	Constraint	Static	SRS 6.3 Security Requirements (SEC-4)
BR-56	Trend-analysis results for queries covering up to 1,000 papers must be displayed within 10 seconds.	Constraint	Dynamic	SRS 6.2 Performance Requirements (PER-2)
BR-57	At least 95% of dashboard pages must finish loading within 4 seconds for a user with a stable connection.	Constraint	Dynamic	SRS 6.2 Performance Requirements (PER-4)
BR-58	The system must support at least 100 concurrent users.	Constraint	Static	SRS 6.2 Performance Requirements (PER-1)

BR-59	PDF report files up to 20 pages must be generated within 15 seconds.	Constraint	Dynamic	SRS 6.2 Performance Requirements (PER-3)
BR-60	The system must be available at least 99% of the time each month, excluding announced maintenance.	Constraint	Dynamic	SRS 6.5 Availability Requirements (AVL-1)
BR-61	If OpenAlex is temporarily unavailable, users must still be able to view previously cached data and analysis results.	Fact	Dynamic	SRS 6.5 Availability Requirements (AVL-2)
BR-62	If PDF report generation is interrupted, the system must record the failure and let the user retry later instead of losing the request.	Constraint	Dynamic	SRS 6.6 Robustness Requirements (ROB-1)
BR-63	If the OpenAlex API returns an error or times out, the system must show a clear message instead of crashing.	Constraint	Dynamic	SRS 6.6 Robustness Requirements (ROB-2)
BR-64	The system must not automatically draw conclusions about a paper's quality or an author's reputation without clearly supporting data.	Constraint	Static	SRS 6.4 Safety Requirements (SAF-2)

K. Search, Discovery & Trending

ID	Rule Definition	Type of Rule	Static or Dynamic	Source
BR-65	A search request cannot combine an exact-value filter with a min/max range filter for the same field (year or citation count); doing so is rejected before OpenAlex is queried.	Constraint	Static	Source code: ErrorCode.INVALID_SEARCH_FILTER_COMBINATION; SearchWorksLookupService.validateExclusiveRangeFilters
BR-66	A works search returns 20 results per page by default and never more than 100 per page.	Constraint	Static	Source code: SearchConstants.DEFAULT_WORKS_PER_PAGE / WORKS_PER_PAGE_LIMIT
BR-67	Searching for papers, viewing a paper's detail, and viewing trending topics/keywords do not require login; only saving/viewing/clearing personal search history requires an authenticated Researcher.	Fact	Static	Source code: SecurityConfig.java (public matcher list for /api/search/summary, /api/search/filters/**, /api/search/works, /api/search/entities, /api/search/trending-topics, /api/search/trending-keywords, /api/papers/**)
BR-68	Trending topics and trending keywords are read from a pre-computed weekly snapshot (dated to the most recent Monday by default), not queried live from OpenAlex; each list defaults to 8 items and never returns more than 20.	Fact	Dynamic	Source code: TopicTrendServiceImpl / KeywordTrendServiceImpl (DEFAULT_LIMIT / MAX_LIMIT)
BR-69	When resolving the titles of a paper's referenced/related works, the system queries OpenAlex in chunks of up to 50 unique work ids; unlike the Bookmark work-type lookup, this enrichment has no graceful-degradation path, so a failed lookup fails the entire paper-detail request.	Constraint	Dynamic	Source code: PaperDetailServiceImpl.fetchTitlesByWorkId

3.2 Use Cases

3.2.1 UC-1 Save and Manage Bookmarked Papers

ID and Name:	UC-1 Save and Manage Bookmarked Papers
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (rev. 2, prod branch)
Primary Actor:	Researcher (authenticated, non-banned user)
Secondary Actors:	None. All paper metadata (title, authors, source, topic, work type, year, and citation count) is supplied directly by the client from data already on screen; the backend does not call any external API as part of this use case.
Description:	A Researcher saves a research paper to their personal bookmark library, optionally organizes bookmarks into named collections (e.g. "Thesis sources"), reviews and filters their bookmark list, moves bookmarks in and out of collections, and removes bookmarks or collections that are no longer needed.
Trigger:	The Researcher clicks the "Bookmark" icon on a paper card or paper detail page, or opens "My Bookmarks" from the personal dashboard.
Preconditions:	PRE-1. The Researcher is logged into SciTrend with a verified, non-banned account. PRE-2. The target paper exists on OpenAlex with a valid openalex_id, and its metadata is already visible to the Researcher (from a prior search or detail view).
Postconditions:	POST-1. A user_bookmark row exists linking the Researcher's account and the paper's openalex_id, storing a snapshot of title, authors, source, topic, work type, publication year, and citation count. POST-2. Optionally, one or more collection_bookmark rows link that bookmark to one or more of the Researcher's bookmark_collection rows. POST-3. Removing a bookmark also removes it from every collection it belonged to. Removing a collection removes only that collection and its bookmark links; the bookmarks themselves are never deleted as a result.
Normal Flow:	**1.0 Save a Paper to Bookmarks** 1. Researcher browses search results, or a topic/author/paper detail page. (see 1.0.E1) 2. Researcher clicks the "Bookmark" icon on a paper. 3. System sends the paper's openAlexId and the metadata already shown on screen to POST /api/bookmarks. 4. System checks whether this Researcher already has a bookmark for this paper. (see 1.0.E2) 5. System creates the user_bookmark record. 6. System marks the icon as "saved" and shows a confirmation, e.g. "Bookmark saved successfully."

Alternative Flows:	**1.1 Create a Collection** 1. Researcher clicks "Create collection" and enters a name (up to 120 characters). 2. System checks the name is not blank and is not already used by this Researcher, case-insensitively. (see 1.1.E1) 3. System creates the bookmark_collection record. Return to step 6 of the Normal Flow. **1.2 Add Bookmark(s) to a Collection** 1. Researcher selects one or more bookmarks and a target collection. 2. System verifies the collection belongs to this Researcher. (see 1.2.E1) 3. System verifies every selected bookmark belongs to this Researcher. (see 1.2.E2) 4. System creates the missing collection_bookmark links; links that already exist are left unchanged. Return to step 6 of the Normal Flow. **1.3 Remove a Bookmark from a Collection** 1. Researcher clicks "Remove" on a bookmark shown inside a collection. 2. System verifies the collection belongs to this Researcher. (see 1.2.E1) 3. System deletes the collection_bookmark link; the bookmark itself remains in the Researcher's overall library. Return to step 6 of the Normal Flow. **1.4 View, Filter, and Sort the Bookmark List** 1. Researcher opens "My Bookmarks." 2. Researcher may filter by collection, title, keyword, topic, source, author, or publication year, and sort by Recent, Oldest, Year, Citation count, or Title. 3. System returns the matching page of bookmarks (12 per page by default, up to 50). Return to step 6 of the Normal Flow. **1.5 Delete a Collection** 1. Researcher clicks "Delete" on a collection. 2. System verifies the collection belongs to this Researcher. (see 1.2.E1) 3. System deletes the collection and every collection_bookmark link that referenced it; the underlying bookmarks are not deleted. Return to step 6 of the Normal Flow. **1.6 Remove a Bookmark Entirely** 1. Researcher clicks the filled "Bookmark" icon again, or clicks "Remove" from the bookmark list or paper detail page. 2. System deletes the user_bookmark record, which also removes it from every collection via cascade. (see 1.6.E1) 3. System reverts the icon to its "unsaved" state and shows a confirmation, e.g. "Bookmark removed successfully." Return to step 6 of the Normal Flow.

Exceptions:	**1.0.E1 Researcher is not logged in (Guest)** 1. System does not perform the bookmark action. 2. System redirects the Researcher to the Login/Registration page. 3. Use case ends. **1.0.E2 Paper is already bookmarked by this Researcher** 1. System detects the existing user_bookmark record (unique on user_id + openalex_id). 2. System returns the existing bookmark unchanged; no duplicate is created and no error is shown. (Un-bookmarking is a separate, explicit action — see 1.6 — not a side effect of clicking "Bookmark" again.) **1.1.E1 Collection name is blank or already exists** 1. System displays "Collection name is required" or "A bookmark collection with this name already exists." 2. No collection is created. **1.2.E1 Collection not found or not owned by this Researcher** 1. System displays "Bookmark collection not found." 2. No change is made. **1.2.E2 One or more selected bookmarks not found or not owned by this Researcher** 1. System rejects the entire request without adding any of the selected bookmarks to the collection. **1.6.E1 Bookmark already removed or not found** 1. System displays "Bookmark not found."
Priority:	High
Frequency of Use:	Expected to be used by most active Academic Users, several times per week, typically right after searching or reading a paper's details.
Business Rules:	BR-3, BR-5, BR-23, BR-24, BR-25, BR-26, BR-27, BR-28, BR-29, BR-30, BR-51, BR-52 (see Business Rules document)
Other Information:	Guests cannot bookmark papers; the action is restricted to authenticated, non-banned users. There is no server-side maximum on the number of papers a Researcher may bookmark or the number of collections they may create; the only cap in the current system is 20 papers per "Export Bookmarked Papers" report request. "Un-bookmarking" is always a distinct user action (clicking the already-saved icon, which the frontend maps to a delete call) — repeating the save action never removes a bookmark.
Assumptions:	The paper metadata snapshot stored with a bookmark is captured at save time from data already displayed to the Researcher; it is not automatically refreshed later if the corresponding OpenAlex record changes.

3.2.2 UC-2 Follow and Manage Followed Authors/Topics

ID and Name:	**UC-2 Follow and Manage Followed Authors/Topics**

Created By:	SciTrend Project Team
Date Created:	12/07/2026 (rev. 2, prod branch)
Primary Actor:	Researcher (authenticated, non-banned user)
Secondary Actors:	None. The display name shown with a follow record is supplied directly by the client at follow time (from data already on screen); the backend does not call the OpenAlex API as part of this use case.
Description:	A Researcher marks a specific author or research topic as "followed" while browsing search results, an author detail page, or a topic detail page, so that SciTrend's scheduled feed synchronization job can surface that entity's latest publications in the Researcher's personalized New Feed. The Researcher can also view a consolidated, filterable list of everything they follow and unfollow entities that are no longer relevant.
Trigger:	The Researcher clicks the "+ Follow" button on an author card, topic card, author detail page, or topic detail page, or opens the "Followed" management page.
Preconditions:	PRE-1. The Researcher is logged into SciTrend with a verified, non-banned account. PRE-2. The target author or topic exists on OpenAlex with a valid ID, and is already visible to the Researcher.
Postconditions:	POST-1. A user_follow row exists (or no longer exists) linking the Researcher's account, target_type ("AUTHOR" or "TOPIC"), and the target's openalex_id. POST-2. The "Follow"/"Following" button state for that author/topic reflects the change everywhere it appears. POST-3. The next scheduled FEED_SYNC run includes or excludes publications associated with the followed/unfollowed entity for this Researcher.
Normal Flow:	**2.0 Follow an Author or Topic** 1. Researcher searches for, or opens the detail page of, an author or a topic. (see 2.0.E1) 2. Researcher clicks the "+ Follow" button. 3. System sends the target type, target OpenAlex ID, and the display name already shown on screen to POST /api/follows. 4. System checks whether the Researcher already follows this target. (see 2.0.E2) 5. System creates the user_follow record with the supplied display-name snapshot. 6. System changes the button label to "Following" and updates its styling to an active state. 7. System displays a confirmation toast, e.g. "You are now following [Author/Topic Name]."

Alternative Flows:	**2.1 View, Filter, and Sort the Followed List** 1. Researcher opens the "Followed" section from the personal dashboard or profile. 2. Researcher may filter by keyword or by target type (Author/Topic), and sort by Recent or Oldest. 3. System returns the matching page of follows (12 per page by default, up to 50), each clearly labeled "Author" or "Topic." (see 2.1.E1) 4. Researcher may click "View Details" on any entry to open that author's or topic's detail page. Return to step 7 of the Normal Flow. **2.2 Unfollow an Author or Topic** 1. Researcher clicks the "Following" button on a card or detail page, or clicks "Unfollow" from the Followed list. 2. System sends a delete request for that target type and target OpenAlex ID. 3. System deletes the matching user_follow record if one exists; if none exists, the request still completes successfully with no error. (see 2.2.E1) 4. System reverts the button to "+ Follow" and shows a confirmation toast, e.g. "You have unfollowed [Author/Topic Name]." 5. The unfollowed entity's publications are excluded starting from the next FEED_SYNC run. Return to step 7 of the Normal Flow.
Exceptions:	**2.0.E1 Researcher is not logged in (Guest)** 1. System does not perform the follow action. 2. System redirects the Researcher to the Login/Registration page. 3. Use case ends. **2.0.E2 Author/topic is already followed by this Researcher** 1. System detects the existing user_follow record (unique on user_id + target_type + target_openalex_id). 2. System returns the existing follow unchanged; no duplicate is created and no error is shown. (Unfollowing is a separate, explicit action — see 2.2 — not a side effect of clicking "Follow" again.) **2.1.E1 Followed list is empty** 1. System displays an empty-state message, e.g. "You aren't following any authors or topics yet." 2. System offers a call-to-action link to the Trending or Discovery page. **2.2.E1 Target was not actually followed** 1. System finds no matching user_follow record. 2. System still reports success; the button/UI state is simply confirmed as "not following."
Priority:	High
Frequency of Use:	Expected to be used by most active Academic Users, typically right after discovering a new author or topic of interest; a core driver of return visits via the New Feed.
Business Rules:	BR-3, BR-5, BR-29, BR-31, BR-32, BR-33, BR-34, BR-35, BR-36 (see Business Rules document)

Other Information:	Following a source/journal is not supported by the current follow module; only AUTHOR and TOPIC targets exist (the earlier SOURCE option has been removed from the schema). The aggregated New Feed itself is covered by a separate use case, "View New Feeds"; it is populated asynchronously by the FEED_SYNC scheduled job, not synchronously at the moment of following. There is no server-side maximum on the number of authors/topics a Researcher may follow.
Assumptions:	The display-name snapshot stored with each follow record is used for fast list rendering and is not automatically refreshed if the author's or topic's name later changes on OpenAlex.

3.2.3 UC-3 Login (Email & Password / Google)

ID and Name:	UC-3 Login (Email & Password / Google)
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Registered User (Researcher or Admin)
Secondary Actors:	Google OAuth2 provider (Google sign-in alternative flow only)
Description:	A registered user authenticates with their email and password, or via their Google account, so that they can access their personal feed, bookmarks, follows, and other authorized features of SciTrend.
Trigger:	The user clicks "Continue to my feed" on the Login page, or clicks the "Continue with Google" button.
Preconditions:	PRE-1. The user already has a registered account. PRE-2. The user is on the Login page.
Postconditions:	POST-1. On success, the system issues a JWT access token (valid 15 minutes by default) and sets a refresh token in an HttpOnly cookie, valid 24 hours by default or 30 days if "Keep me signed in" is checked. POST-2. The user is redirected to their personal feed/workspace. POST-3. On failure, no token is issued and the user remains on the Login page with an error message.
Normal Flow:	**3.0 Login with Email and Password** 1. Registered User navigates to the Login page. 2. User enters email and password. 3. User clicks "Continue to my feed." 4. System validates the credentials against the stored account data. (see 3.0.E1) 5. System confirms the account is not banned. (see 3.0.E2) 6. System confirms the account's email has been verified. (see 3.0.E3) 7. System issues a JWT access token and a refresh token in an HttpOnly cookie. 8. System redirects the user to their personal feed/workspace.

Alternative Flows:	**3.1 Toggle Password Visibility** 1. User clicks the "eye" icon inside the password field. 2. System toggles the password between masked characters and plain text. Return to step 3 of the Normal Flow. **3.2 Keep Me Signed In** 1. User checks "Keep me signed in on this device" before submitting. 2. At step 7 of the Normal Flow, system issues a refresh token valid for 30 days instead of the 24-hour default. The JWT access token's 15-minute lifetime is unaffected. Return to step 8 of the Normal Flow. **3.3 Login via Google (existing account only)** 1. User clicks "Continue with Google" instead of steps 2–3 of the Normal Flow. 2. System redirects the browser to Google's consent screen; User authorizes a Google account. (see 3.3.E1) 3. Google returns the account's verified email, given name, and family name. (see 3.3.E2) 4. System checks whether an account already exists for that email and whether it is banned. (see 3.3.E3, 3.3.E4) 5. Since an account already exists, system links the Google identity to it (sets googleLinked = true, emailVerified = true, backfills any missing name/avatar), issues a refresh token, and redirects the browser to the frontend's OAuth2 callback page carrying that token. 6. The frontend immediately calls the token-exchange endpoint with the received token to obtain the JWT access token and to have the real HttpOnly refresh-token cookie set. Return to step 8 of the Normal Flow. **3.4 Navigation Links** 1. User clicks "Forgot password?" and is redirected to the password recovery (OTP) flow. 2. User clicks "Create a free account" and is redirected to the Registration page (UC-4). 3. User clicks the Home icon and is redirected to the Landing Page. Use case ends without login.

Exceptions:	**3.0.E1 Incorrect credentials** 1. System detects that the email does not match any account, the password is wrong, or the account has no local password at all (a Google-only account attempting local login). 2. System displays a single generic error, "Incorrect email or password," without revealing which case applied. Return to step 2 of the Normal Flow. **3.0.E2 Banned account** 1. System detects the account is marked as banned. 2. System rejects the login and displays a message indicating the account has been suspended. Use case ends. **3.0.E3 Unverified account** 1. System detects the account's email has not been verified. 2. System displays a message prompting the user to verify their email and does not issue a token. Use case ends. **3.3.E1 Google authentication cancelled or failed** 1. User closes the Google consent window, clicks Cancel, or a network/API error occurs. 2. System returns the user to the Login page; if a network/API error occurred, system displays "Login failed. Please try again later." Otherwise no error is shown. Return to step 2 of the Normal Flow. **3.3.E2 Google reports the email as unverified** 1. Google's email_verified attribute is not true for the authorizing account. 2. System rejects the sign-in and returns the user to the Login page with an error. Use case ends. **3.3.E3 Google account is banned** 1. System detects the matching local account is banned. 2. System rejects the sign-in and redirects to the Login page with a suspension message, the same outcome as 3.0.E2. Use case ends. **3.3.E4 No account exists for this Google email** 1. System does not log the user in and does not silently create an account. 2. System instead starts the Google registration flow (see UC-4, Alternative Flow "Register via Google"). This use case ends without a login.
Priority:	High
Frequency of Use:	Expected to be used by every active Academic User, an average of 3–5 times per week (per SRS 2.2 User Classes).
Business Rules:	BR-3, BR-5, BR-6, BR-7, BR-8, BR-9, BR-10, BR-11, BR-12, BR-13, BR-14, BR-21, BR-22 (see Business Rules document)

Other Information:	The system allows multiple concurrent sessions across different devices, since logging in does not revoke previously issued refresh tokens; however, logging out revokes every refresh token for that account, signing the user out everywhere at once, not just on the current device (BR-11, BR-12). The team's confirmed Business Rules table does not currently define an account-lockout rule for repeated failed login attempts; none is enforced in this use case. For Google sign-in, session establishment is a two-step handshake: the OAuth2 redirect carries a one-time refresh token in the URL, which the frontend must immediately exchange via the backend for the real access token and HttpOnly cookie.
Assumptions:	The user has previously completed email verification (for local accounts) or has a Google account whose email Google reports as verified.

3.2.4 UC-4 Register (Email & Password / Google)

ID and Name:	**UC-4 Register (Email & Password / Google)**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Guest (unregistered visitor)
Secondary Actors:	Gmail SMTP (sends the verification email; local registration only); Google OAuth2 provider (Google sign-up alternative flow only)
Description:	A Guest creates a personal account using an email address and password, or by continuing with an existing Google account, so that they can access Researcher features such as bookmarking, following, and personalized feeds. For local accounts, the account is unusable until its email address is verified; for Google sign-ups, the Guest additionally chooses a password to finish creating the account, and the account is considered verified immediately.
Trigger:	The Guest clicks "Create account and begin exploring" on the Registration page, or clicks "Continue with Google" (the same button used for UC-3 Login).
Preconditions:	PRE-1. The user is not currently logged in. PRE-2. The user is on the Registration page (or arrives via the Google redirect described in Alternative Flow 4.1).
Postconditions:	POST-1 (local). A new users row is created with a unique email, hashed password, role = RESEARCHER, is_email_verified = false, and is_google_authenticated = false. POST-2 (local). A verification link valid for 24 hours is generated and emailed to the registered address. POST-3 (local). The user is redirected to the Login page with a confirmation message; no session is issued yet. POST-4 (Google). A new users row is created with role = RESEARCHER, is_email_verified = true, is_google_authenticated = true, and a password hash chosen by the Guest; a JWT access token and refresh-token cookie are issued immediately, logging the Guest in without a separate verification step.

Normal Flow:	**4.0 Register with Email and Password** 1. Guest navigates to the Registration page. 2. Guest enters First Name, Last Name, Email, Password, and Confirm Password. 3. Guest clicks "Create account and begin exploring." 4. System checks that the email is validly formatted and not already registered, and that First/Last name are present (up to 255 characters each). (see 4.0.E1) 5. System creates the account with role RESEARCHER and status "Unverified." 6. System generates a verification link valid for 24 hours and sends it to the registered email via Gmail SMTP. (see 4.0.E2) 7. System redirects the Guest to the Login page and displays "Register success. Please check your email to verify your account."
Alternative Flows:	**4.1 Register via Google OAuth2** 1. Guest clicks "Continue with Google" instead of step 2 of the Normal Flow. 2. System redirects to Google's consent screen; Guest authorizes a Google account. (see 4.1.E1) 3. Google returns the account's verified email, given name, and family name. (see 4.1.E2) 4. System checks whether an account already exists for that email. (see 4.1.E3) 5. Since no account exists, system issues a one-time Google signup token, valid for 10 minutes, and redirects the browser to the frontend's "Complete registration" page carrying that token. 6. Frontend loads the Guest's email, first name, and last name for display via the signup-preview endpoint. (see 4.1.E4) 7. Guest enters and confirms a password, between 10 and 100 characters. (see 4.1.E5) 8. System validates the signup token, creates the account with is_email_verified = true and is_google_authenticated = true, marks the signup token as used, and immediately issues a JWT access token and refresh-token cookie. (see 4.1.E6) Use case ends successfully with the Guest already logged in (steps 5–7 of the Normal Flow are skipped; no separate email verification is required). **4.2 Toggle Password Visibility** 1. Guest clicks the "eye" icon inside the Password or Confirm Password field. 2. System toggles the field between masked characters and plain text. Return to step 3 of the Normal Flow.

Exceptions:	**4.0.E1 Invalid or duplicate email** 1. System detects the email format is invalid, or the email already belongs to an existing account. 2. System displays the corresponding inline error (e.g., "Invalid email format" or "The email has been exist.") and does not create the account. Return to step 2 of the Normal Flow. **4.0.E2 Verification email fails to send** 1. The account has already been created but the email fails to send. 2. System displays a warning: "Account created, but we couldn't send the verification email. Please request a new one." **4.1.E1 Google authentication cancelled or failed** 1. Guest closes the Google consent window, clicks Cancel, or a network/API error occurs. 2. System returns the Guest to the Registration page; no account is created. **4.1.E2 Google reports the email as unverified** 1. Google's email_verified attribute is not true for the authorizing account. 2. System rejects the sign-up attempt with an error; no account or signup token is created. **4.1.E3 Google email already registered** 1. System finds an existing account with that email. 2. System does not create a duplicate account or a signup token. This attempt is instead handled as an existing-account Google sign-in (see UC-3, Alternative Flow "Login via Google"); this use case ends without registering a new account. **4.1.E4 Signup token invalid, already used, or expired** 1. System cannot find the token, or finds it already used, or finds it expired (more than 10 minutes old). 2. System displays "The registration token is invalid," "This registration token has already been used," or "This registration token has expired," as applicable. 3. Guest must restart Google sign-up from step 1. **4.1.E5 Password / Confirm Password mismatch or too short/long** 1. System detects Password ≠ Confirm Password, or either field is outside the 10–100 character range. 2. System displays "Password confirmation does not match" or the corresponding length error. Return to step 7 of the Alternative Flow. **4.1.E6 Email became registered before completion** 1. Another registration completed for that email while this signup token was still pending. 2. System rejects completion with "The email has been exist." Use case ends.
Priority:	High
Frequency of Use:	Expected once per new user; several new registrations per day during active recruitment/onboarding periods.

Business Rules:	BR-1, BR-2, BR-3, BR-4, BR-6, BR-13, BR-14, BR-15, BR-16, BR-17, BR-18, BR-22 (see Business Rules document)
Other Information:	Unlike the Google-completion flow, local email/password registration does not currently enforce a minimum password length or re-check that Password and Confirm Password match on the backend; both fields are only required to be non-blank server-side. The frontend is expected to enforce the match and any strength rules, but this is not currently re-validated by the API. The team may want to add this validation to close the gap (BR-2). A local account cannot log in until its email is verified (BR-3); a Google-completed account is considered verified immediately and can log in right away. "Continue with Google" is one shared entry point for both Login (UC-3) and Register (UC-4): the backend decides which use case applies based on whether an account already exists for the Google email.
Assumptions:	The Guest has access to the email inbox used for local registration and can receive the verification email within a few minutes; for Google sign-up, the Guest completes the password step within the 10-minute signup-token window.

3.2.5 UC-5 Search Papers

ID and Name:	**UC-5 Search Papers**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Guest or Researcher (GET /api/search/works is public and does not require login)
Secondary Actors:	OpenAlex API — every search request is answered by querying OpenAlex live (no local cache of results); the system only adds its own filter validation, pagination, and an internal "trending" flag on top of the OpenAlex response.
Description:	A user searches for scientific papers by keyword and/or a combination of filters (year, citation count, work type, open access, subfield, author, institution, country, source, award, ORCID indexing, trending status), sorts the results, and pages through them. A logged-in Researcher may additionally save a completed search to their personal search history for later reuse.
Trigger:	The user types a keyword and/or selects filters on the Search page and submits the search, or opens "Search" from the navigation bar.
Preconditions:	PRE-1. None — Search is available to unauthenticated Guests as well as logged-in Researchers. PRE-2. OpenAlex must be reachable for results to be returned (see 5.0.E3).
Postconditions:	POST-1. A page of matching works is displayed to the user, sourced live from OpenAlex. POST-2. If the user is logged in and chooses to save the search, a search_history row is created for that user (see 5.1).

Normal Flow:	**5.0 Search Works by Keyword and Filters** 1. User opens the Search page. 2. User enters a keyword and/or selects filters: publication year (as an exact year or a from/to range — not both), citation count (as an exact value or a min/max range — not both), work type, open access only, subfield, author, institution, country, source, award, ORCID-indexed status, and/or trending mode (none / keyword / topic / both). 3. User chooses a sort order (Relevance, Citation count, or Publication date; ascending or descending) and, optionally, a page number and page size. (see 5.0.E1) 4. System sends the request to GET /api/search/works. 5. System validates that no filter's exact-value and range forms are combined for the same field. (see 5.0.E1) 6. System queries OpenAlex with the resolved filters, sort, and pagination. (see 5.0.E2, 5.0.E3) 7. System returns the matching page of works (title, authors, source, year, citation count, work type, open access flag, topic) to the user.
Alternative Flows:	**5.1 Save a Completed Search to History (Researcher only)** 1. After viewing results, the logged-in Researcher chooses to save the search (e.g. it is saved automatically by the frontend once results load). 2. System sends the query text to POST /api/search/history for the current Researcher. (see 5.1.E1) 3. System stores the search as a search_history row for that Researcher. Return to step 7 of the Normal Flow. **5.2 View, Reuse, or Clear Recent Search Suggestions (Researcher only)** 1. Researcher opens the search box; system calls GET /api/search/history/recent to show up to 7 recent searches by default, optionally filtered by keyword. 2. Researcher may click a suggestion to reuse it (return to step 4 of the Normal Flow with that keyword), delete a single entry (DELETE /api/search/history), or clear all history (DELETE /api/search/history/all). **5.3 Browse Filter Option Lists** 1. User opens a filter dropdown (e.g. Author, Institution, Source, Award, Subfield). 2. System calls GET /api/search/filters/{filterKey}/options with an optional keyword to page through the available values for that filter, sourced from OpenAlex. **5.4 Search Authors or Topics Instead of Works** 1. User switches the search scope to "Authors" or "Topics." 2. System calls GET /api/search/entities with the corresponding entityType and returns matching authors or topics instead of works.

Exceptions:	**5.0.E1 An exact-value filter is combined with a range filter for the same field** 1. System rejects the request with "Year exact cannot be combined with Year from/to" or "Citation exact cannot be combined with Citation min/max," as applicable, before contacting OpenAlex. 2. No results are returned; the user must correct the filter combination. **5.0.E2 OpenAlex returns no matches** 1. System returns an empty result page. 2. UI shows "No papers found" with a suggestion to broaden the search. **5.0.E3 OpenAlex is unavailable or times out** 1. The request fails after automatic retries. 2. System shows an error message ("Search is temporarily unavailable, please try again shortly") instead of crashing. **5.1.E1 User is not logged in** 1. The save-history request is rejected as unauthenticated (search history is not in the public endpoint list). 2. The search results themselves remain visible; only the save action is blocked.
Priority:	High
Frequency of Use:	Very high — this is the primary entry point for both Guests and Researchers, used many times per session.
Business Rules:	BR-51, BR-63, BR-65, BR-66, BR-67 (see Business Rules document)
Other Information:	Unlike Bookmarks, Search results are not cached locally: every search call queries OpenAlex live, so result freshness depends entirely on OpenAlex's own data currency. A works search page defaults to 20 results and cannot request more than 100 per page. The "trending" filter (trendingMode) flags works that also appear in the current weekly trending-topics/keywords snapshot (see UC-7); it does not change how the underlying OpenAlex query itself is scored.
Assumptions:	The scoped domain/field restriction built into every search query (Scientific/Computer Science-oriented fields, per SearchConstants) is assumed to match the product's intended subject-matter focus and is not user-configurable.

3.2.6 UC-6 View Paper Detail

ID and Name:	**UC-6 View Paper Detail**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Guest or Researcher (GET /api/papers/{workId} is public and does not require login)
Secondary Actors:	OpenAlex API — the entire paper detail record, plus the titles of its referenced and related works, are fetched live from OpenAlex; nothing about the paper is stored locally.

Description:	A user opens the full detail page for a single paper, viewing its complete OpenAlex metadata (abstract, authors, source, topic, citation count, open access status, etc.) along with human-readable titles for the works it references and the works OpenAlex considers related to it.
Trigger:	The user clicks a paper's title/card from search results, a bookmark list, a followed topic/author's feed, or a trending list.
Preconditions:	PRE-1. None — viewing a paper's detail does not require login. PRE-2. The caller has a valid OpenAlex work id (openAlexId), normally obtained from a prior search, bookmark, or feed listing.
Postconditions:	POST-1. The full paper detail, including a referenced_work_details list and a related_work_details list (each work id paired with its resolved title, or the raw id if no title could be resolved), is displayed to the user.
Normal Flow:	1. User clicks on a paper title/card. 2. System sends GET /api/papers/{workId}. 3. System requests the full work record from OpenAlex using the given workId. (see 6.E1) 4. System collects the referenced_works and related_works id lists from the OpenAlex response, normalizes each id (stripping the OpenAlex URL prefix, if present) and removes duplicates. 5. System queries OpenAlex again, in chunks of up to 50 unique work ids at a time, to resolve a display title for each referenced/related work id. (see 6.E2) 6. System attaches a referenced_work_details and a related_work_details list (id + resolved title, falling back to the raw id if no title was found) to the response. 7. System returns the enriched paper detail to the user.
Alternative Flows:	None — this is a single, read-only lookup with no branching user choices. From the detail page the user may separately trigger UC-1 (bookmark this paper) or UC-2 (follow its authors/topic), which are documented as their own use cases.
Exceptions:	**6.E1 The requested workId does not exist on OpenAlex** 1. OpenAlex returns a 404 for the work lookup, which the system does not retry. 2. System returns "OpenAlex entity not found." 3. UI shows "This paper could not be found." **6.E2 OpenAlex is unavailable or times out, either for the main record or for a reference/related-work title lookup** 1. The request fails after automatic retries. 2. Unlike Bookmark's work-type resolution (see UC-1), this use case has no graceful-degradation path: if any of the chunked reference/related-work title lookups ultimately fails, the whole paper detail request fails rather than rendering with unresolved titles. 3. System shows an error message instead of crashing.
Priority:	High
Frequency of Use:	Very high — this is the primary way both Guests and Researchers read about an individual paper, reached from search, bookmarks, feed, and trending lists.
Business Rules:	BR-51, BR-63, BR-67, BR-69 (see Business Rules document)

Other Information:	The paper detail response is a pass-through, lightly enriched OpenAlex record rather than a SciTrend-owned domain object; SciTrend does not persist a local copy of the paper's full detail.
Assumptions:	The requested workId is assumed to be an OpenAlex work id in the standard "W12345678" form; the endpoint does not attempt to resolve a DOI or other external identifier into a work id.

3.2.7 UC-7 View Trending Topics and Keywords

ID and Name:	**UC-7 View Trending Topics and Keywords**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Guest or Researcher (GET /api/search/trending-topics and /api/search/trending-keywords are both public)
Secondary Actors:	None. Unlike Search and Paper Detail, this use case does not call OpenAlex at request time — it reads from SciTrend's own topic_trend_snapshot / keyword_trend_snapshot tables, which are populated ahead of time by a separate scheduled synchronization job.
Description:	A user views a ranked list of the currently trending research topics and/or keywords, computed from the most recent weekly snapshot, to get a quick sense of what is gaining attention without running a full search.
Trigger:	The user opens the "Trending" section of the dashboard or home page.
Preconditions:	PRE-1. None — viewing trending topics/keywords does not require login. PRE-2. At least one weekly snapshot must already exist in the system (populated by the scheduled sync job); if none exists yet for the resolved week, the list is simply empty.
Postconditions:	POST-1. A ranked list of up to the requested number of trending topics or keywords for the resolved snapshot week is displayed, each with its identifier, name, associated field, work count, and citation count.
Normal Flow:	**7.0 View Trending Topics** 1. User opens the Trending section. 2. System sends GET /api/search/trending-topics, optionally with a specific snapshotDate and/or a result limit. 3. If no snapshotDate is given, system resolves it to the most recent Monday on or before today. (see 7.0.E1) 4. System clamps the requested limit to a default of 8 if none/invalid is given, and to a maximum of 20. 5. System reads the top trending topics for that snapshot week from the internal topic_trend_snapshot table. 6. System returns the ranked list (topic id, name, field id, work count, citation count) to the user.

Alternative Flows:	**7.1 View Trending Keywords** 1. User switches the Trending view to "Keywords." 2. System sends GET /api/search/trending-keywords with the same snapshotDate/limit rules as 7.0. 3. System returns the ranked list of trending keywords (keyword id, keyword text, field id, work count, citation count) for that snapshot week. Return to step 6 of the Normal Flow. **7.2 Drill Down from a Trending Item** 1. User clicks a trending topic/keyword. 2. System routes the user into UC-5 (Search Papers) pre-filtered to that topic/keyword, or into the fuller Trend Analysis Dashboard (out of scope for this use case).
Exceptions:	**7.0.E1 No snapshot exists yet for the resolved week** 1. The underlying query returns zero rows. 2. System returns an empty list (snapshotDate plus an empty items array), not an error. 3. UI shows "No trending data available for this period yet."
Priority:	Medium
Frequency of Use:	Frequent — checked regularly by returning Researchers as a quick pulse-check, and by Guests browsing the home page.
Business Rules:	BR-67, BR-68 (see Business Rules document)
Other Information:	Trending topics/keywords are refreshed only as often as the scheduled sync job runs (weekly, snapshot dated to the preceding Monday); this view can lag behind the very latest OpenAlex publications by design, in contrast to Search and Paper Detail which are always live.
Assumptions:	"Trending" here refers strictly to the simple weekly topic/keyword ranking exposed under /api/search/trending-*; it is a distinct, simpler feature from the Trend Analysis Dashboard's momentum/radar/heatmap scoring, which is out of scope for this use case.

3.2.8 UC-8 Ban / Unban a User Account

ID and Name:	**UC-8 Ban / Unban a User Account**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Administrator
Secondary Actors:	Email delivery service — a notification email is sent to the affected account when it is banned or unbanned.
Description:	An Administrator suspends ("bans") a Researcher account so it can no longer sign in, or restores ("unbans") a previously suspended account, from the Manage Users screen.
Trigger:	The Administrator clicks "Ban" or "Unban" next to a user in the Manage Users list or on that user's detail page.
Preconditions:	PRE-1. The caller is authenticated and holds the ADMIN role (enforced at the security-filter level: /api/admin/** requires ROLE_ADMIN). (see 8.E1) PRE-2. The target userId refers to an existing account.

Postconditions:	POST-1 (Ban). The target account's banned flag is set to true; all of that account's active refresh tokens are revoked, signing it out of every device; a ban notification email is sent; a UserBannedEvent is published. POST-2 (Unban). The target account's banned flag is set to false; a restoration notification email is sent; a UserUnbannedEvent is published (no token action is needed, since a banned account has no active sessions to begin with).
Normal Flow:	**8.0 Ban a User** 1. Administrator opens Manage Users (see UC-10) and selects a Researcher account. 2. Administrator clicks "Ban" and confirms. 3. System sends PATCH /api/admin/users/{userId}/ban. 4. System verifies the Administrator is not targeting their own account. (see 8.E2) 5. System verifies the target account exists. (see 8.E3) 6. System verifies the target account does not itself hold the ADMIN role. (see 8.E4) 7. System verifies the target account is not already banned. (see 8.E5) 8. System sets banned = true on the target account. 9. System revokes every active refresh token belonging to the target account, signing it out everywhere. 10. System publishes a UserBannedEvent, which triggers a ban-notification email to the target account. 11. System confirms "Account banned successfully" to the Administrator.
Alternative Flows:	**8.1 Unban a User** 1. Administrator clicks "Unban" on a currently banned account and confirms. 2. System sends PATCH /api/admin/users/{userId}/unban. 3. System repeats guard steps 4–6 of the Normal Flow (not-self, exists, not an admin account). 4. System verifies the target account is currently banned. (see 8.E6) 5. System sets banned = false on the target account. 6. System publishes a UserUnbannedEvent, which triggers a restoration-notification email to the target account. 7. System confirms "Account unbanned successfully" to the Administrator.

Exceptions:	**8.E1 Caller is not an authenticated Administrator** 1. The request is rejected before reaching the admin service (403 Forbidden, or 401 if not authenticated at all). **8.E2 Administrator targets their own account** 1. System rejects with "Admin cannot perform this action on their own account." **8.E3 Target user does not exist** 1. System rejects with "User not found." **8.E4 Target account holds the ADMIN role** 1. System rejects with "Admin accounts cannot be banned or unbanned." **8.E5 Target account is already banned (ban flow)** 1. System rejects with "This account has already been banned." **8.E6 Target account is not currently banned (unban flow)** 1. System rejects with "This account is not banned."
Priority:	High
Frequency of Use:	Low — an occasional moderation action, used only when abuse or policy violations are identified.
Business Rules:	BR-5, BR-11, BR-47 (as corrected), BR-48, BR-49, BR-50 (see Business Rules document)
Other Information:	A banned account is denied login through both the local and Google sign-in paths (see UC-3) and loses all active sessions immediately upon being banned. There is no separate "delete account" action in the current admin module; ban/unban is the only account-suspension mechanism.
Assumptions:	The ADMIN role is assumed to be assigned out-of-band (e.g. directly in the database or by another Administrator through a future user-management feature); the current admin module has no "promote to Admin" action.

3.2.9 UC-9 View Admin Dashboard

ID and Name:	**UC-9 View Admin Dashboard**
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Administrator
Secondary Actors:	None directly — all figures are computed from SciTrend's own database (users, search_history, topic/subfield tables, sync-job logs); no live OpenAlex call is made to render the dashboard.
Description:	An Administrator views a system-health and usage overview: user growth, banned-account counts, API usage figures, catalog size (topics/subfields), the latest data-synchronization timestamp, and the currently configured sync schedules.
Trigger:	The Administrator opens the Admin Dashboard page.
Preconditions:	PRE-1. The caller is authenticated and holds the ADMIN role (/api/admin/** requires ROLE_ADMIN). (see 9.E1)

Postconditions:	POST-1. The dashboard is populated with the current statistics cards, ban summary, API usage figures, and sync schedule; no data is modified by viewing it.
Normal Flow:	1. Administrator opens the Admin Dashboard. 2. System sends GET /api/admin/dashboard/statistics. 3. System computes and returns nine statistic cards: total users (with week-over-week delta), active trend signals (with week-over-week delta), banned users (with month-over-month delta), API calls this month (with percentage change vs. the previous month), API calls today, total configured daily API credit (defaulting to 100,000 if not overridden), total subfields, total topics (with fields count and period delta), and the timestamp of the latest data synchronization. 4. System displays the nine cards to the Administrator.
Alternative Flows:	**9.1 View Ban Summary** 1. System also calls GET /api/admin/users/ban-summary. 2. System returns the count and percentage of active vs. banned accounts. 3. Dashboard displays this as a supporting widget. **9.2 View API Usage Breakdown** 1. System calls GET /api/admin/dashboard/api-calls/top-users to list the top 5 accounts by API usage this month. 2. System calls GET /api/admin/dashboard/api-calls/usage-over-time to chart daily API usage for the last 7 days. 3. Both figures are derived from the search_history table as a proxy for API usage; there is no separate, dedicated API-call log. **9.3 View Synchronization Schedule** 1. System calls GET /api/admin/config/feed-sync-cron and GET /api/admin/config/dashboard-sync-cron. 2. System displays the currently configured cron schedules for the personalized-feed sync job and the statistics/dashboard sync job. 3. This is a read-only display in the current implementation; there is no endpoint to change these schedules from the Admin Dashboard.
Exceptions:	**9.E1 Caller is not an authenticated Administrator** 1. The request is rejected before reaching the admin service (403 Forbidden, or 401 if not authenticated at all).
Priority:	Medium
Frequency of Use:	Frequent for Administrators — expected to be checked routinely (e.g. daily) to monitor system health and growth.
Business Rules:	BR-47 (as corrected), BR-70 (see Business Rules document)
Other Information:	"API usage" throughout the dashboard is a proxy metric derived from recorded search activity (search_history), not a dedicated request-metering system; it should be read as an approximation of load, not an exact external API call count.
Assumptions:	The daily API credit figure (100,000 by default) is assumed to reflect an externally negotiated OpenAlex rate limit and is configured, not computed.

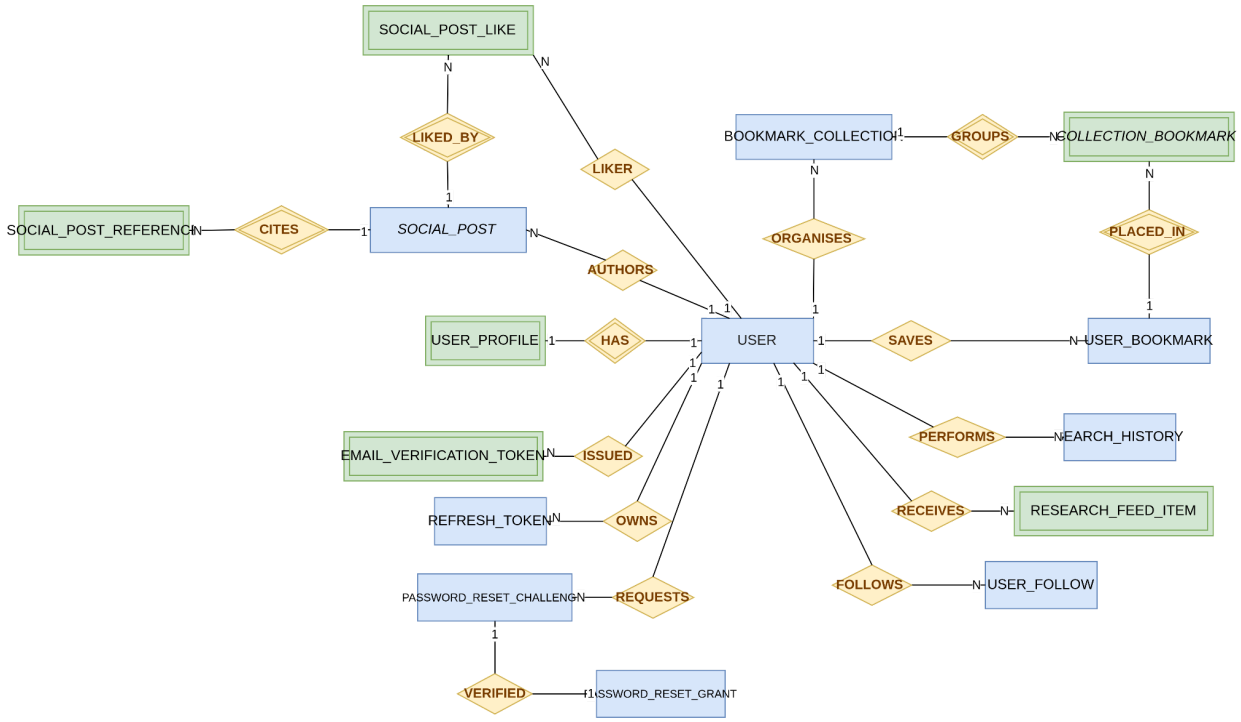
3.2.10 UC-10 Manage Users

ID and Name:	UC-10 Manage Users
Created By:	SciTrend Project Team
Date Created:	12/07/2026 (prod branch)
Primary Actor:	Administrator
Secondary Actors:	None.
Description:	An Administrator browses the full list of registered accounts, filters/sorts it, and drills into a single account's profile and activity (follows, bookmarks, search history) to review its standing before deciding whether to take a moderation action (see UC-8, Ban/Unban).
Trigger:	The Administrator opens the "Manage Users" section of the admin panel.
Preconditions:	PRE-1. The caller is authenticated and holds the ADMIN role (/api/admin/** requires ROLE_ADMIN). (see 10.E1)
Postconditions:	POST-1. The Administrator has viewed a page of the user list and/or a specific user's detail; no account data is modified by this use case on its own.
Normal Flow:	**10.0 Browse the User List** 1. Administrator opens Manage Users. 2. System sends GET /api/admin/users with a page number, page size (10 by default, capped at 100), and a sort order (Most recent, Oldest, Email A–Z, or Email Z–A; "Most recent" by default). 3. System returns a page of accounts, each annotated with its role, ban status, and follow counts (topics followed, authors followed). 4. System displays the list to the Administrator.
Alternative Flows:	**10.1 View a Single User's Detail** 1. Administrator clicks on an account in the list. 2. System sends GET /api/admin/users/{userId}. (see 10.E2) 3. System returns the account's core fields, profile (institution, department, country), and activity summary (topics followed, authors followed, bookmark count, search history count). 4. System displays the account detail page. **10.2 View a User's Search History** 1. From the account detail page, Administrator opens "Search History." 2. System sends GET /api/admin/users/{userId}/search-history with a page number and size. (see 10.E2) 3. System returns that account's paginated search history. **10.3 Take a Moderation Action** 1. From the list or the detail page, Administrator clicks "Ban" or "Unban." 2. System proceeds with UC-8 (Ban / Unban a User Account).
Exceptions:	**10.E1 Caller is not an authenticated Administrator** 1. The request is rejected before reaching the admin service (403 Forbidden, or 401 if not authenticated at all). **10.E2 The target userId does not exist** 1. System rejects with "User not found."
Priority:	Medium

Frequency of Use:	Moderate — used by Administrators whenever they need to look up or review a specific account, or periodically audit the user base.
Business Rules:	BR-47 (as corrected), BR-70 (see Business Rules document)
Other Information:	Manage Users is read-only in the current implementation beyond the Ban/Unban action; there is no edit-profile-on-behalf-of-user or delete-account action available to Administrators.
Assumptions:	None beyond those already stated for UC-8 and UC-9.

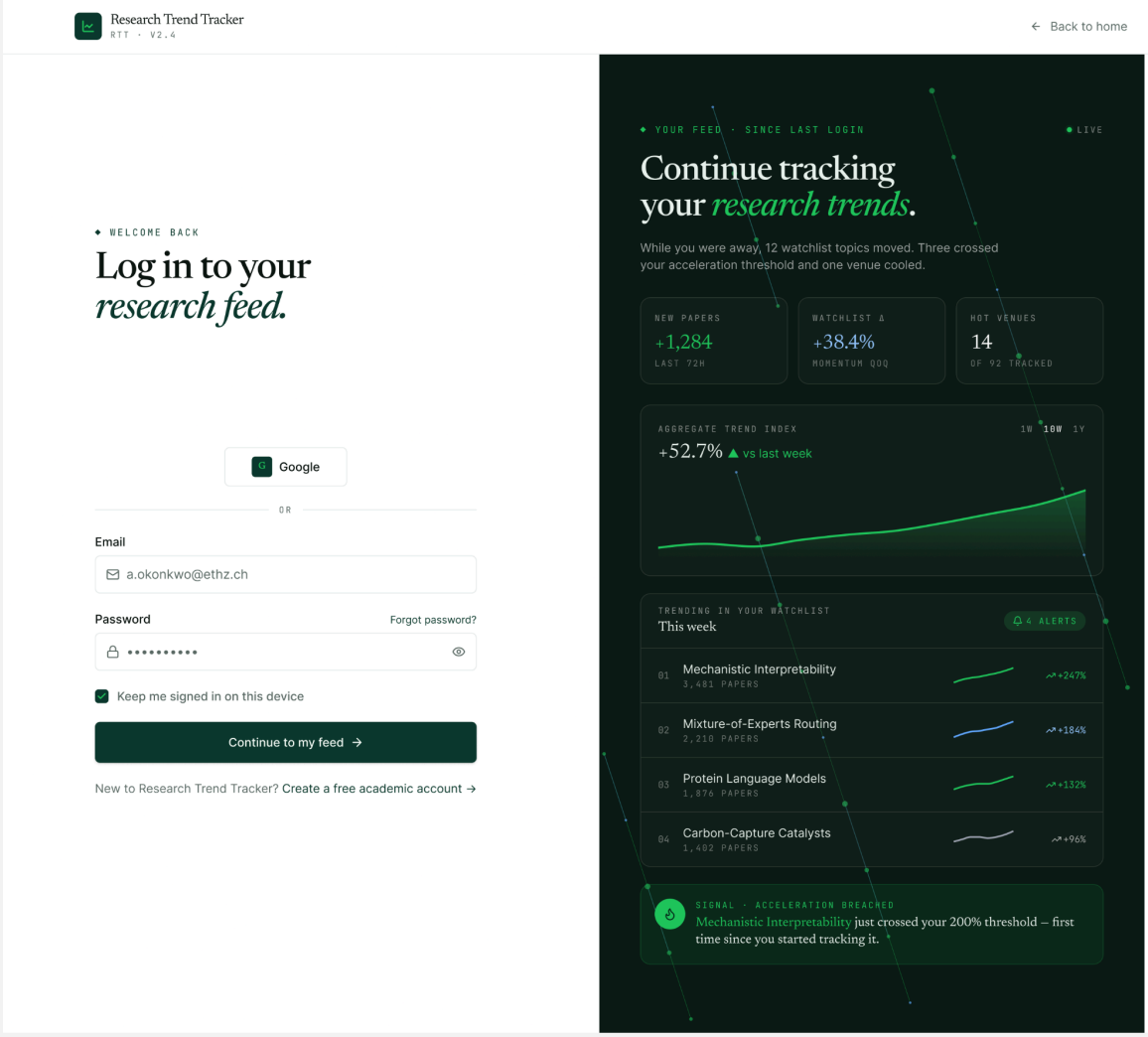
4. Data Requirements

4.1 Logical Data Model



4.2 Data Dictionary

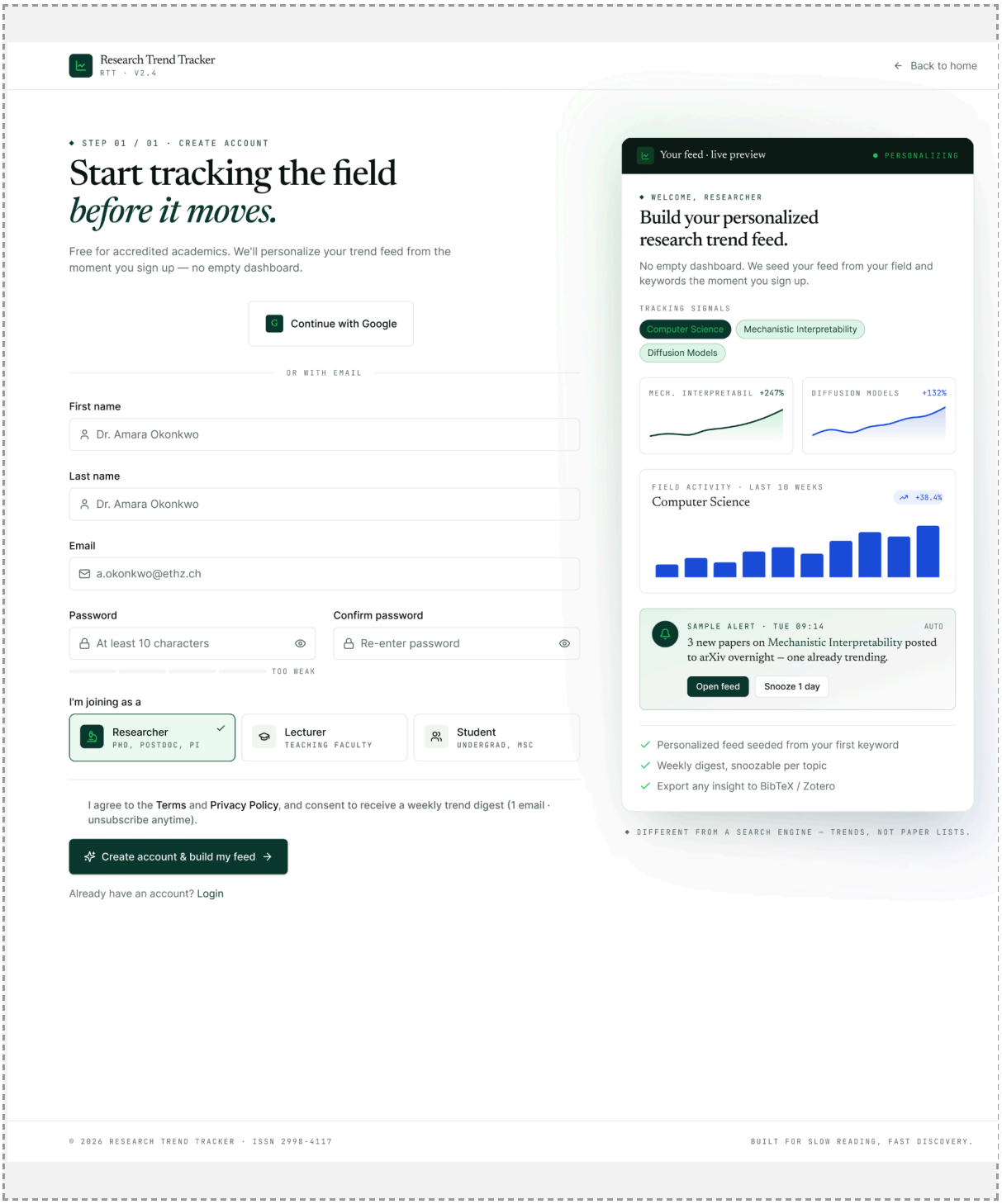
4.2.1 Screen 1 — Login (Email & Password / Google)



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Email — Email address used to sign in with a local account.	String	Yes	Yes	N/A	N/A	N/A
Password — Account password for local login.	String	No	Yes	N/A	N/A	N/A
Remember me — If checked, the issued refresh token is valid for 30 days instead of the default 24 hours.	Boolean	No	No	N/A	N/A	false

"Login with Google" is a button, not a data-entry field. All three fields above are one-time inputs on a form, not saved records — "Editable?" is marked N/A because the concept of editing a stored value does not apply here.

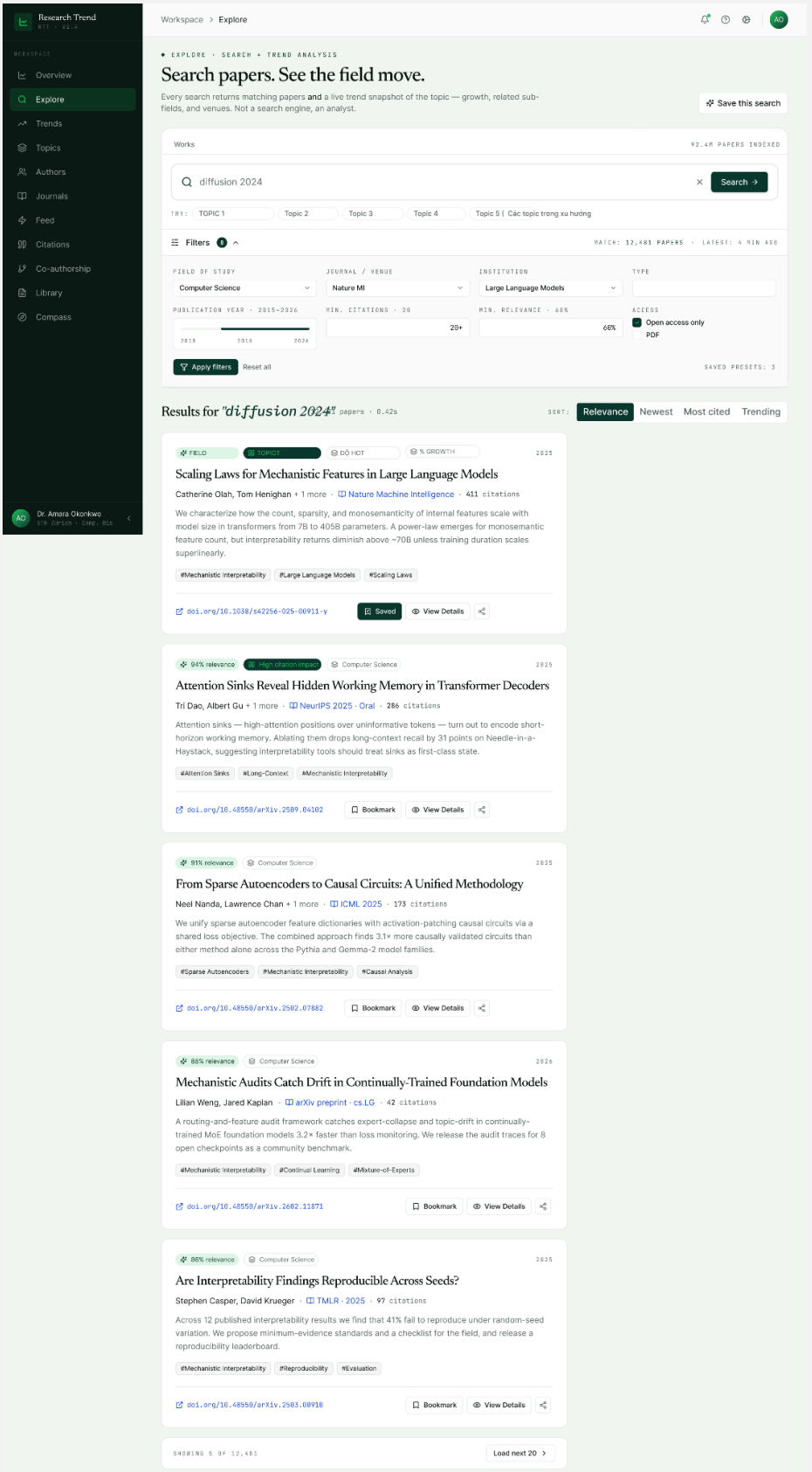
4.2.2 Screen 2 — Register (Email & Password / Google)



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Email — Email address used to create the local account; becomes the sign-in identifier. Unique across all accounts; cannot be changed after the account is created.	String	Yes	Yes	No	N/A	N/A
First Name — Given name of the user.	String	No	No	Yes	N/A	N/A
Last Name — Family name of the user.	String	No	No	Yes	N/A	N/A
Password — Password for the new local account; stored hashed, changed later via a separate flow. The backend does not currently enforce a minimum length or a match check on this screen.	String	No	Yes	Yes	N/A	N/A
Confirm Password — Re-typed password on the Register screen, to catch typos; input only, not persisted.	String	No	Yes	N/A	N/A	N/A
Password (Google step) — Password set on the follow-up screen shown the first time a user signs in with Google.	String	No	Yes	Yes	10-100 characters	N/A
Confirm Password (Google step) — Re-typed password on the Google-completion screen; must match Password above.	String	No	Yes	N/A	10-100 characters	N/A

"Register with Google" is a button that starts the Google flow; the two Google-step fields only appear on the follow-up "complete registration" screen shown after a first-time Google sign-in, not on the main Register screen.

4.2.3 Screen 3 — Search Papers

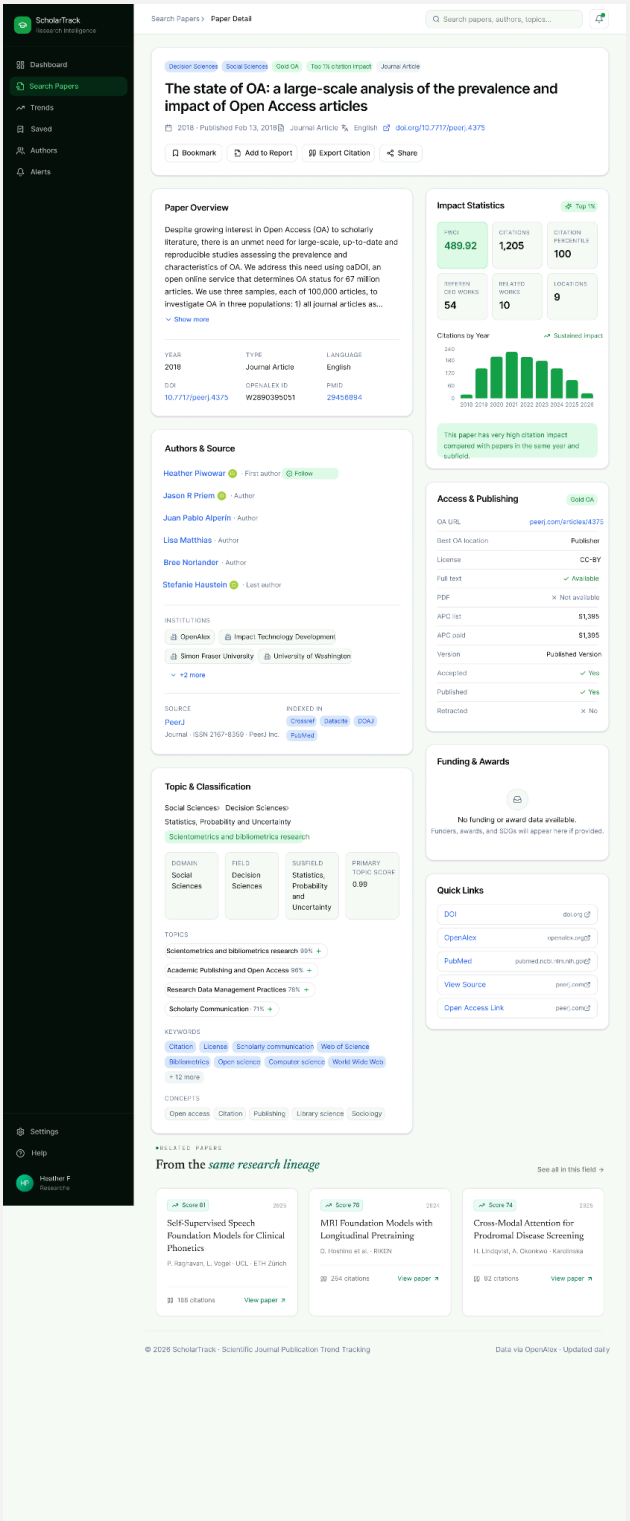


Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Keyword — Free-text search query.	String	No	No	Yes	N/A	empty
Year filter mode — Whether the Year filter below is an exact year or a from/to range.	Enum	No	No	Yes	N/A	range
Year from / to — Publication year range; usable only when Year filter mode = range.	Integer	No	No	Yes	min year 100	empty
Year exact — A single exact publication year; usable only when Year filter mode = exact.	Integer	No	No	Yes	N/A	empty
Work type — One or more OpenAlex work types (e.g. article, preprint).	List<String>	No	No	Yes	N/A	empty (all types)
Open Access only — Restrict results to open-access works.	Boolean	No	No	Yes	N/A	false
Subfield — One or more OpenAlex subfield ids to filter by.	List<String>	No	No	Yes	N/A	empty
Author — One or more OpenAlex author ids to filter by.	List<String>	No	No	Yes	N/A	empty
Institution — One or more OpenAlex institution ids to filter by.	List<String>	No	No	Yes	N/A	empty
CSV available — Restrict results to works with a full-text CSV.	Boolean	No	No	Yes	N/A	false
Country — One or more country codes to filter by.	List<String>	No	No	Yes	N/A	empty
Citation filter mode — Whether the Citation filter below is an exact value or a min/max range.	Enum	No	No	Yes	N/A	range
Citation min / max — Citation-count range; usable only when Citation filter mode = range.	Integer	No	No	Yes	min 0	empty
Citation exact — A single exact citation count; usable only when Citation filter mode = exact.	Integer	No	No	Yes	N/A	empty
Source — One or more OpenAlex source (journal/venue) ids to filter by.	List<String>	No	No	Yes	N/A	empty
Award / Funder — One or more OpenAlex funder/award ids to filter by.	List<String>	No	No	Yes	N/A	empty
ORCID indexed — Filter authors by whether they are indexed by ORCID.	Enum	No	No	Yes	N/A	empty

Trending mode — Flags whether results should be limited to works also appearing in the current trending snapshot.	Enum	No	No	Yes	N/A	none
Sort by — Result ordering field: relevance, citation, or published date.	Enum	No	No	Yes	N/A	relevance
Sort direction — Ascending or descending order.	Enum	No	No	Yes	N/A	desc
Page — Current results page number.	Integer	No	No	Yes	N/A	1
Results per page — Number of results shown per page.	Integer	No	No	Yes	max 100	20

All rows above are filter/query inputs, not stored records — "Editable?" therefore means "can the user change this control on screen." Year exact and Year from/to are mutually exclusive, as are Citation exact and Citation min/max — selecting one clears the other on the real screen.

4.2.4 Screen 4 — Paper Detail

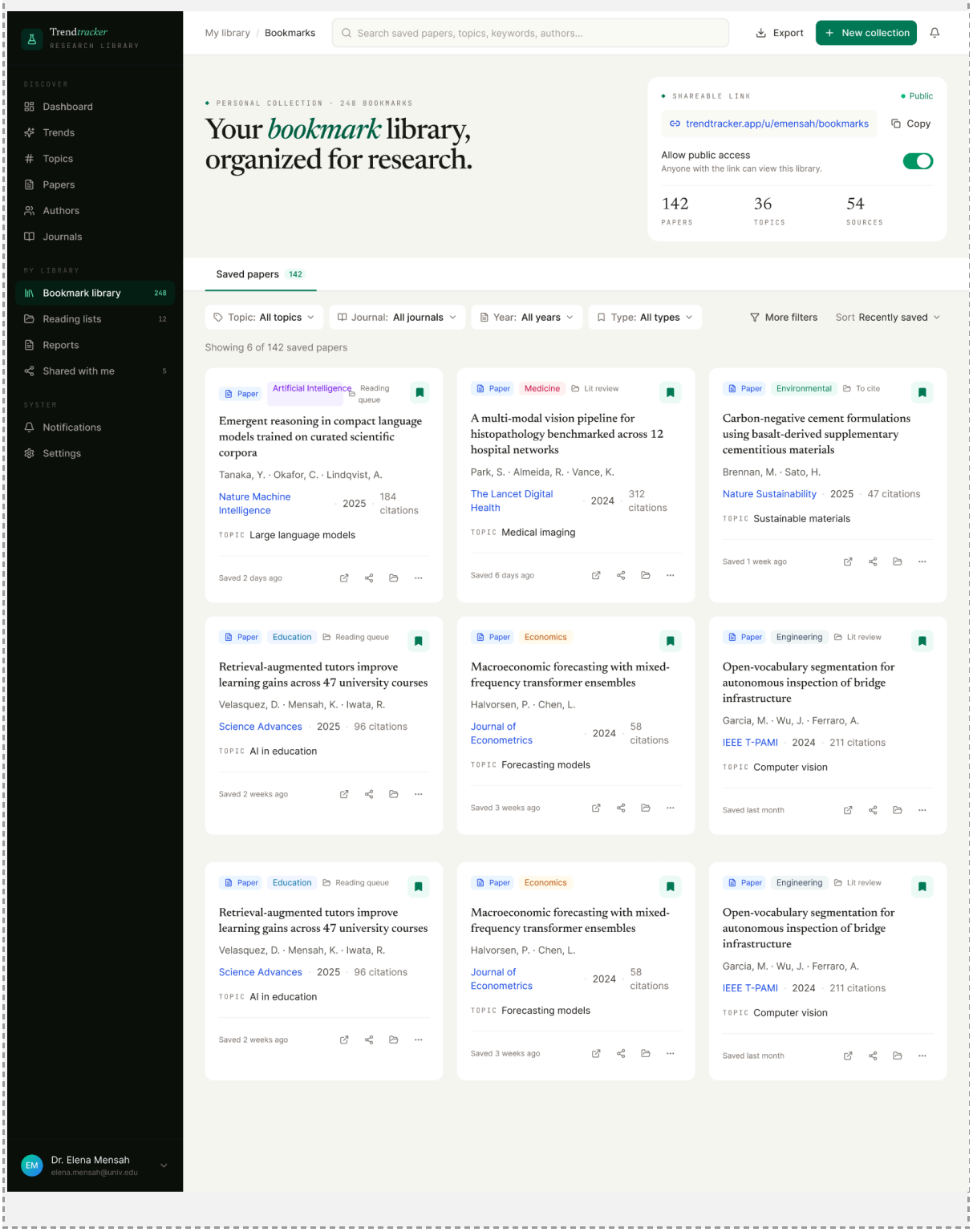


Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value

Title — Display title of the paper.	String	No	N/A	No	N/A	N/A
Authors — List of author names/ids on the paper.	List<Object>	No	N/A	No	N/A	N/A
Source — Journal / venue the paper was published in.	Object	No	N/A	No	N/A	N/A
Publication Year — Year the paper was published.	Integer	No	N/A	No	N/A	N/A
Citation Count — Number of times the paper has been cited.	Integer	No	N/A	No	N/A	N/A
Work Type — OpenAlex work type (article, preprint, etc.).	String	No	N/A	No	N/A	N/A
Open Access — Whether the paper is open access, and its OA status/URL.	Object	No	N/A	No	N/A	N/A
Abstract — Reconstructed abstract text of the paper.	String	No	N/A	No	N/A	N/A
Primary Topic — The paper's primary OpenAlex topic/field/domain classification.	Object	No	N/A	No	N/A	N/A
DOI — Digital Object Identifier of the paper.	String	No	N/A	No	N/A	N/A
Referenced Work Details — List of works this paper cites, each resolved to an id + title.	List<Object>	No	N/A	No	N/A	N/A
Related Work Details — List of works OpenAlex considers related, each resolved to an id + title.	List<Object>	No	N/A	No	N/A	N/A

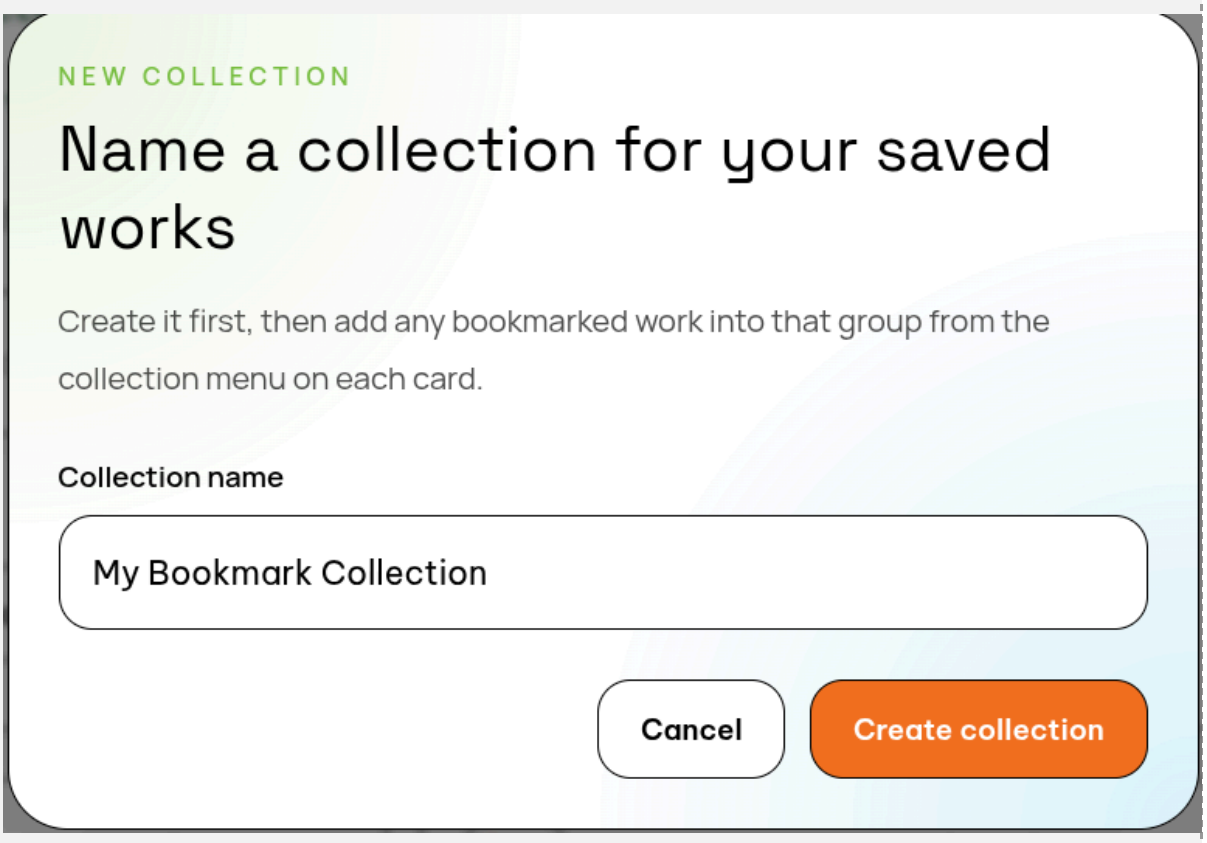
Paper Detail is a lightly-enriched pass-through of the raw OpenAlex work record (a generic key/value object), not a fixed SciTrend-owned schema — the rows above cover the fields the frontend is known to render; the full OpenAlex work object may contain additional fields not listed here.

4.2.5 Screen 5a — Bookmark Item



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
OpenAlex ID — The bookmarked paper's OpenAlex work id; unique per user.	String	Yes	Yes	No	N/A	N/A
Title — Snapshot of the paper's title, captured automatically at bookmark time.	String	No	Yes	No	N/A	N/A
Authors — Snapshot of the paper's authors, captured automatically at bookmark time.	String / List	No	Yes	No	N/A	N/A
Source — Snapshot of the paper's source/venue at bookmark time.	String	No	No	No	N/A	N/A
Topic — Snapshot of the paper's primary topic at bookmark time.	String	No	No	No	N/A	N/A
Work Type — Snapshot of the paper's work type; resolved from OpenAlex automatically if not supplied by the client.	String	No	No	No	N/A	unresolved if OpenAlex lookup fails
Publication Year — Snapshot of the paper's publication year.	Integer	No	No	No	N/A	N/A
Citation Count — Snapshot of the paper's citation count at bookmark time.	Integer	No	No	No	N/A	N/A

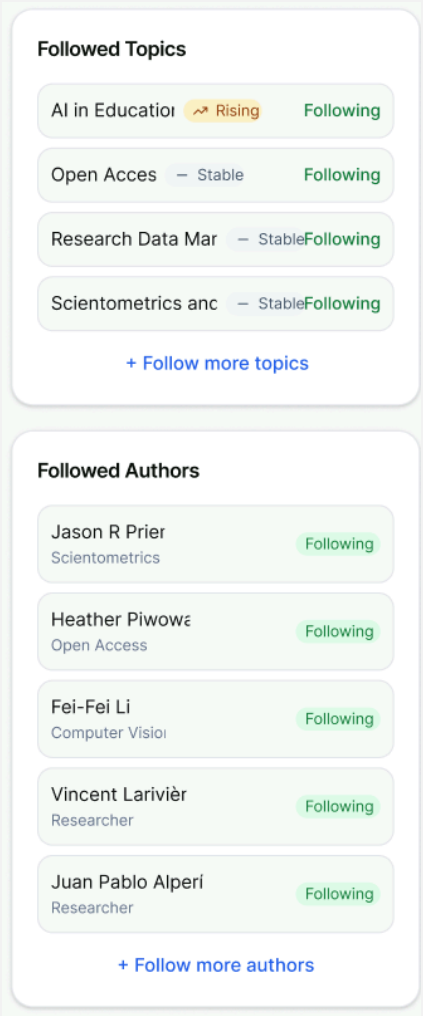
4.2.6 Screen 5b — Bookmark Collection



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Collection Name — User-chosen name for a bookmark collection (e.g. "Thesis sources"); unique per user, case-insensitive.	String	Yes	Yes	Yes	1-120 characters	N/A

A bookmark's metadata (title, authors, source, etc.) is a snapshot captured at save time from data already on screen — it is not automatically refreshed if the underlying OpenAlex record later changes. Only Work Type may be silently resolved after the fact via an OpenAlex lookup.

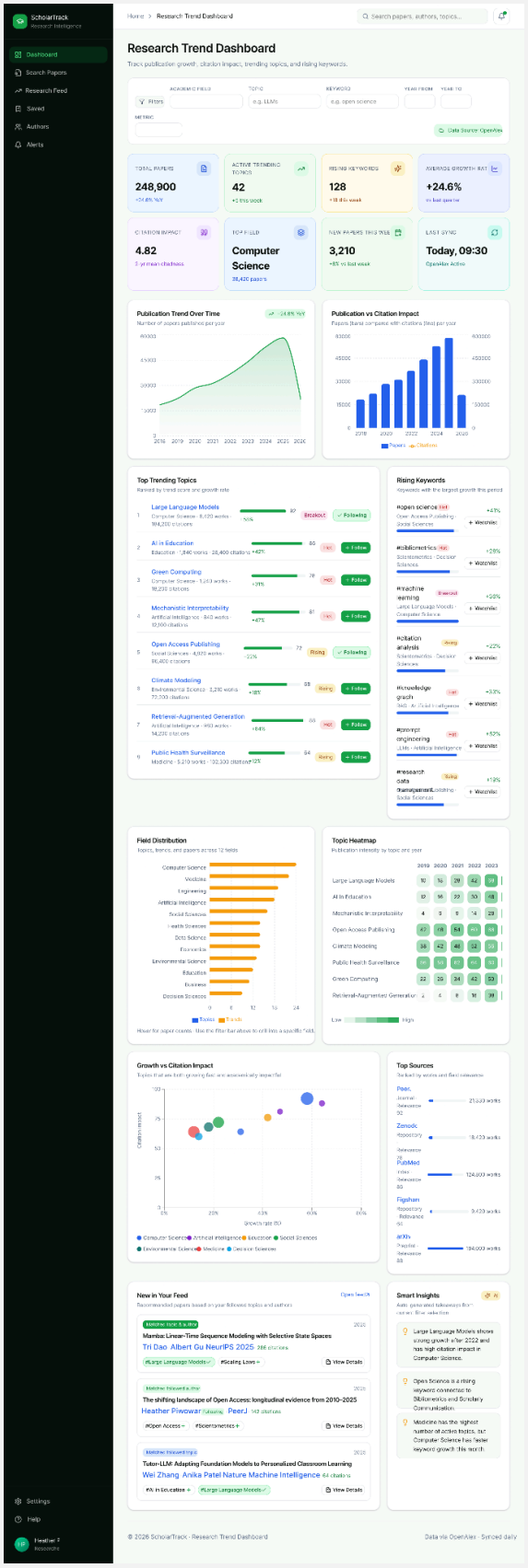
4.2.7 Screen 6 — Follow (Authors / Topics)



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Target Type — Whether the followed target is an Author or a Topic.	Enum	No	Yes	No	N/A	N/A
Target OpenAlex ID — The OpenAlex id of the followed author or topic; unique per user + target type.	String	Yes	Yes	No	N/A	N/A
Display Name — Name shown for the followed author/topic on screen; supplied by the client, not fetched from OpenAlex.	String	No	Yes	No	N/A	N/A

Display Name is supplied by the client at follow time (from data already visible on screen) and is not independently verified or refreshed by the backend.

4.2.8 Screen 7 — Trending (Topics / Keywords)



Description	Data Type	Unique?	Required?	Editable?	Size / Length	Default Value
Topic ID / Keyword ID — Identifier of the trending topic or keyword.	String	No	N/A	No	N/A	N/A
Name / Keyword — Display name of the trending topic, or the keyword text.	String	No	N/A	No	N/A	N/A
Field ID — OpenAlex field id the topic/keyword is associated with.	Integer	No	N/A	No	N/A	N/A
Works Count — Number of works counted for this topic/keyword in the current snapshot week.	Integer	No	N/A	No	N/A	N/A
Citations Count — Number of citations counted for this topic/keyword in the current snapshot week.	Integer	No	N/A	No	N/A	N/A

All fields on this screen are read-only and come from a pre-computed weekly snapshot table, not a live OpenAlex query.

4.3 Report

Information Field	Description
Report ID	RTA-RPT-1
Report Title	Export Bookmarked Papers
Report Purpose	Allows Researchers to select bookmarked papers and export their metadata as a dataset for analysis, use in spreadsheets or data-processing pipelines, or sharing.
Priority	Medium
Report Users	Researchers
Data Sources	The user's bookmark records stored in the system database and paper metadata retrieved from the OpenAlex API.
Frequency and Disposition	The report is generated on demand when a Researcher opens the Report page, selects between 1 and 20 bookmarked papers, chooses the metadata fields and export format, and clicks **Export**. The file is downloaded to the user's device in CSV or JSON format. The report contains static data captured at the time of export.

Latency	The system shall generate and initiate the report download within 10 seconds under normal operating conditions.
Visual Layout	The report is exported as a structured dataset. CSV files use a tabular layout, while JSON files use an object-and-array structure.
File Header Information	The exported file shall contain the export date, search query, total number of selected papers, list of selected fields, and export format. In CSV files, this information appears at the beginning of the file. In JSON files, it is stored in the metadata object. No file footer is required.
Report Body	The system supports 15 metadata fields: Title, Author, Year, Citation Count, DOI, Type, Open Access, Field, Domain, Subfield, Institution, Country, Topic, Keyword, and Abstract. Users may select one or more fields for export. Only the selected fields shall appear in the generated file.
CSV File Structure	After the file header information, the CSV file shall contain one row of column headings followed by one data row for each selected paper.
JSON File Structure	The JSON file shall contain a metadata object and a papers array. Each item in the papers array represents one paper and contains only the metadata fields selected by the user.
Selection Criteria	Only papers selected by the Researcher from their personal bookmark list shall be included in the report. Each export request shall include no more than 20 papers.
Sort Criteria	The report shall preserve the order of papers displayed on the Report page unless the user applies a different sorting option before exporting.
End-of-Report Indicator	None.
Interactivity	Before exporting, the Researcher may select or deselect papers, choose metadata fields, and select CSV or JSON as the export format. The downloaded file is not interactive.
Security Access Restrictions	Researchers may view and export only papers contained in their own bookmark lists. The system shall verify the authenticated user's ownership of the selected bookmarks before generating the report.
Error Handling	The system shall not generate a report when no paper, metadata field, or export format has been selected. If file generation fails, the system shall display an appropriate error message and shall not download an incomplete file.

4.4 Data Integrity, Retention, and Disposal

The SciTrend system shall preserve the integrity, consistency, and availability of research metadata and user-generated data throughout their lifecycle. The system shall retain data only for the period necessary to support system functions and shall securely dispose of obsolete or unnecessary data in accordance with the defined data-retention policy.

4.4.1 DI-1: Research Metadata Integrity

The system shall store research metadata retrieved from external academic data sources without modifying its original meaning. Stored metadata shall include, where available, the title, abstract, publication year, DOI, authors, institutions, citation count, research topics, and publication source.

4.4.2 DI-2: Duplicate Data Prevention

The system shall prevent duplicate research-work records by validating unique identifiers, such as the OpenAlex Work ID or DOI, before storing retrieved metadata.

4.4.3 DI-3: Data Consistency

The system shall ensure that updates to research metadata, bookmarks, follows, posts, and user profiles are performed consistently so that users do not encounter conflicting or outdated information.

4.4.4. DI-4: Cached Metadata Retention

The system shall retain cached research metadata obtained from external academic APIs for a maximum period of **30 days**, after which the data may be refreshed or replaced during subsequent synchronization.

4.4.5 DI-5: User Activity Retention

The system shall retain user-generated activity records, including search history, bookmarks, followed topics, and generated reports, only for the period required to support system functionality and user experience.

4.4.6 DI-6: Social Hub Content Retention

The system shall retain Social Hub posts and user comments until they are deleted by their owners or removed by an administrator in accordance with system policies.

4.4.7 DI-7: Secure Data Disposal

When a user permanently deletes personal content, including posts, bookmarks, or reports, the system shall remove the content from normal user access and shall prevent future retrieval through the original resource identifier.

4.4.8 DI-8: Account Deletion

When a user account is permanently deleted, the system shall remove or anonymize associated personal information in accordance with the system's data-retention policy while preserving any data required for legal, auditing, or statistical purposes.

4.4.9 DI-9: Synchronization Integrity

The system shall ensure that scheduled synchronization with external academic data sources does not create inconsistent, incomplete, or partially updated metadata records.

4.4.10. DI-10: Backup and Recovery

The system shall maintain periodic backups of persistent system data to support recovery in the event of data corruption, accidental deletion, or system failure.

5. External Interface Requirements

5.1 User Interfaces

The SciTrend system provides a responsive web-based user interface that enables users to search research works, analyze publication trends, manage personal collections, interact with the Social Hub, and administer the system. The user interface shall emphasize usability, accessibility, and consistency across all supported web browsers.

5.1.1. UI-1: Web-based Research Dashboard

The system shall provide a web-based dashboard that allows users to:

- search research works using keywords and filters;
- browse search results and research details;
- view publication trend charts and analytics;
- manage bookmarks and followed topics;
- access personalized recommendations and Social Hub features.

5.1.2 UI-2: Consistent User Interface

The system shall provide a consistent user interface across all pages by maintaining uniform navigation menus, typography, color schemes, icons, buttons, and interaction behaviors.

5.1.3 UI-3: Responsive User Interface

The system shall support responsive layouts for desktop, tablet, and mobile web browsers while maintaining usability and readability.

5.1.4 UI-4: User Guidance

The system shall provide help text, placeholders, tooltips, or contextual guidance for major functions including search filters, trend charts, bookmarking, Social Hub operations, and report exporting.

5.1.5 UI-5: Keyboard Accessibility

The system shall support keyboard navigation for major user operations, including:

- navigating menus;
- searching research works;
- selecting filters;
- viewing research details;
- bookmarking papers;
- interacting with Social Hub posts;
- exporting reports.

The system shall allow these operations without requiring a pointing device.

5.1.6 UI-6: Data Visualization

The system shall present research trends using interactive visualizations such as line charts, bar charts, ranking lists, and topic statistics that support user interaction (e.g., hovering, zooming, or filtering where applicable).

5.1.7 UI-7: Feedback and Status Messages

The system shall provide appropriate visual feedback for user actions, including loading indicators, success notifications, warning messages, validation errors, and system errors.

5.1.8 UI-8: Error Presentation

The system shall display understandable error messages whenever an operation cannot be completed, including search failures, authentication failures, unavailable external services, or invalid user input.

5.1.9 UI-9: Confirmation for Destructive Actions

The system shall require user confirmation before performing irreversible operations, including deleting posts, removing bookmarks, unfollowing topics, or deleting user accounts.

5.1.10. UI-10: Accessibility

The system shall provide sufficient color contrast, readable typography, descriptive labels for interactive controls, and compatibility with common accessibility features to improve usability for users with different abilities.

5.2 Software Interfaces

5.2.1 SI-1: OpenAlex API

The OpenAlex API is the primary source of academic data for the SciTrend system.

- **SI-1.1:** The system shall retrieve research works from OpenAlex based on user-entered keywords and supported filters, including topics, authors, institutions, and publication years.

- **SI-1.2:** The system shall retrieve available metadata, including title, abstract, publication year, DOI, authors, institutions, citation count, work type, open-access status, field, domain, subfield, country, topics, and keywords.
- **SI-1.3:** The system shall use data retrieved from OpenAlex to support research-work searches, work-detail viewing, trend analysis, bookmarking, and report exporting.
- **SI-1.4:** If the OpenAlex API is unavailable or returns an error, the system shall notify the user and shall not display incomplete or invalid research data.
- **SI-1.5:** The system shall avoid unnecessary duplicate requests and comply with the request limits of the OpenAlex API.

5.2.2 SI-2: Google OAuth2 Service

Google OAuth2 is used to support user registration and authentication through Google accounts.

- **SI-2.1:** The system shall allow users to register or sign in using a Google account.
- **SI-2.2:** The system shall redirect users to Google for authentication and receive the authentication result through the configured callback endpoint.
- **SI-2.3:** The system shall validate the authentication information returned by Google before creating an authenticated user session.
- **SI-2.4:** The system shall retrieve only the user information permitted by Google, such as the user's name, email address, and profile image.
- **SI-2.5:** If the Google authentication process is cancelled, rejected, or fails, the system shall not create an authenticated session and shall display an appropriate message.
- **SI-2.6:** The system shall not expose the Google client secret or other authentication credentials to the frontend.

5.2.3 SI-3: Gmail SMTP Service

The Gmail SMTP service is used to deliver system-generated emails.

- **SI-3.1:** The system shall send an account-verification email after a user completes the registration process.
- **SI-3.2:** The system shall send an OTP or password-recovery instructions when a user requests a password reset.
- **SI-3.3:** The system shall send a notification email when an Admin bans or unbans a user account.
- **SI-3.4:** Email content shall be generated from predefined email templates.
- **SI-3.5:** The system shall send emails only to the email address associated with the corresponding user account.
- **SI-3.6:** If Gmail SMTP is unavailable or email delivery fails, the system shall return or display an appropriate error message for the related operation.
- **SI-3.7:** A failure to send a ban or unban notification email shall not reverse a successfully completed account-status change.

5.2.4 SI-4: DiceBear API

The DiceBear API is used to generate default avatars for user accounts.

- **SI-4.1:** The system shall generate a default avatar URL when a newly created user account does not have a profile image.
- **SI-4.2:** The generated avatar shall be associated with the corresponding user profile.
- **SI-4.3:** The avatar shall be generated from a stable identifier so that the same user receives a consistent default avatar.

- **SI-4.4:** If DiceBear is unavailable, the system shall use a predefined fallback avatar or continue creating the account without an automatically generated avatar.
- **SI-4.5:** Failure to generate an avatar shall not prevent the user account from being created.

5.3 Hardware Interfaces

No specific hardware interfaces have been identified for the system. The application is a software-based web application that operates entirely within standard web browsers on end-user devices (such as personal computers, laptops, tablets, and smartphones).

The system utilizes standard user input/output peripherals (e.g., keyboard, mouse, and display monitor) provided by the host operating system, and does not require integration with any specialized hardware devices (such as barcode scanners, biometrics readers, or custom IoT sensors).

No hardware interfaces have been identified.

5.4 Communications Interfaces

5.4.1 CI-1: Frontend–Backend Communication

- **CI-1.1:** The SciTrend frontend shall communicate with the backend through HTTPS.
- **CI-1.2:** Data exchanged between the frontend and backend shall use the JSON format unless otherwise specified.
- **CI-1.3:** The system shall use RESTful API endpoints for communication between the client and server.
- **CI-1.4:** Sensitive information, such as authentication tokens and user credentials, shall not be transmitted through unencrypted HTTP connections.
- **CI-1.5:** If a request fails, times out, or receives no response, the system shall display an appropriate error message to the user.

5.4.2 CI-2: Email Communication

- **CI-2.1:** The system shall send an account verification email after a user registers using an email address and password.
- **CI-2.2:** The system shall send an OTP or password-reset instructions when a user requests password recovery.
- **CI-2.3:** The system shall send a notification email when an Admin bans or unbans a user account.
- **CI-2.4:** Emails shall be sent to the email address associated with the corresponding user account.
- **CI-2.5:** Email content shall provide sufficient information for the user to understand the purpose of the notification.
- **CI-2.6:** Verification links, password-reset links, and OTP codes shall expire after the validity period configured by the system.
- **CI-2.7:** The system shall not include passwords or other sensitive authentication information in email messages.

5.4.3 CI-3: External API Communication

- **CI-3.1:** The system shall communicate with the OpenAlex API, Google OAuth2 service, and DiceBear API through HTTPS.
- **CI-3.2:** The system shall send and receive data using the format required by each external service.
- **CI-3.3:** The system shall validate responses received from external services before processing or displaying their data.
- **CI-3.4:** If an external service is unavailable, returns an invalid response, or exceeds the configured timeout, the system shall handle the error and notify the user when the failure affects the requested operation.
- **CI-3.5:** API keys, client secrets, and other private configuration values shall be stored securely and shall not be exposed to the frontend.

6. Quality Attributes

6.1 Usability Requirements

USE-1: Users can perform a research trend search by entering keywords and selecting a timeframe within a single screen.
USE-2: 90% of new users must be able to create their first trend analysis report without requiring direct guidance or tutorials.

6.2 Performance Requirements

PER-1: The system shall support a minimum of 100 concurrent users at the same time.
PER-2: Trend analysis results must be displayed within 10 seconds for queries that involve up to 1,000 research papers.
PER-3: CSV report files must be generated within 15 seconds for reports that are up to 20 pages long.
PER-4: 95% of the dashboard pages must finish loading within 4 seconds when the user has a stable Internet connection.

6.3 Security Requirements

SEC-1: Users must log in to the system to save their search history, create reports, and download previously generated reports.
SEC-2: Each user is only allowed to view and download the reports they created themselves, with the exception of the Administrator who has full access.
SEC-3: All network requests containing user information or passwords must be transmitted securely via HTTPS.
SEC-4: The system must verify user permissions before allowing any user to delete reports or view personal analysis history.

6.4 Safety Requirements

SAF-1: The system must display a clear warning stating that the trend analysis results are for reference purposes only and are highly dependent on the data provided by OpenAlex.
SAF-2: The system shall not automatically make conclusions about the quality of a research paper or an author's reputation without clear supporting data.

6.5 Availability Requirements

AVL-1: The system must be available and operational at least 99% of the time each month, excluding previously announced maintenance periods.
AVL-2: If the external OpenAlex API is temporarily unavailable, the system shall still allow users to view data and analysis reports that have been saved in the local database cache.

6.6 Robustness Requirements

ROB-1: If the CSV report generation process is interrupted (e.g., due to a network loss), the system must save the failed status and allow the user to easily retry creating the report later.
ROB-2: If the OpenAlex API returns an error or times out, the system must display a clear and understandable error message to the user instead of crashing the application.
ROB-3: If a running trend analysis loses its client-side connection before finishing, the system must allow the user to view the completed results later, provided the data was successfully processed and saved.

Appendix

A. Member Evaluation Table

Name	Score (%)	Note
Hoàng Thị Khánh Linh (SE190721)	23	Leader
Nguyễn Huy Phong (SE201436)	18	
Hồ Lê Thiên An (SE201566)	18	
Nguyễn Đình Hải Đăng (SE201605)	18	
Huỳnh Hữu Phúc (SE201562)	23	
	100%	